

Contents

The spec2rtl Methodology	3
0. The route, end to end	3
Part I — What a specification is	6
1. The problem, and the tool underneath	6
2. The thesis: what the specification is, and what it is not	7
3. The causal layer, and how it compiles — forms, not a compiler . . .	8
4. Semantic memory: the representation policy	9
Part II — The language	10
5. What the language is for, and what it refuses to be	10
5.1 The constructs at a glance	11
5.2 The whole language in one small block	12
5.3 Prior art this language borrows from, deliberately	14
6. The two files	15
7. Lexical conventions	16
8. The signature file	17
8.0a A port's shape, and what a subscript means	18
9. The core notation: complete grammar	23
10. The constructs, one by one	26
10.1 @state — semantic state the specification names	26
10.2 @define — giving a semantic concept a name	28
10.3 @behavior — how the abstract machine operates	28
10.4 @property — what must be true of every execution	29
10.5 @scenario — what must remain possible	29
10.6 @assume — what the environment promises	30
11. Time	30
12. The frame rule	32
12.0a A window holds ONE value, and that is checked	32
12.1 What the frame is keyed BY	33
13. Well-formedness — what the compiler checks before anything is proven	34
14. The obligations	36
15. Worked example: the miss queue	36
16. What is borrowed from SVA, and what deliberately is not	38
17. What is deliberately not in the language	39
Part III — The contract, and how it is proven	39
18. The central idea: properties on the outside, linked to the inside . .	40
18.1 What the ports can already say	40
18.2 What the ports cannot say, and the wrong way to fix it . . .	40
18.3 Windows: seeing the design's own state, without copying it .	41
19. Induction in normal form	42
19.1 The problem induction solves: you cannot walk there	42

19.2	The move: abstract the initial state	42
19.3	The normal form	43
19.4	Two k's, named apart	44
20.	Scenarios: catching vacuity without scripts	44
20.1	The vacuity problem	44
20.2	Constrained abstract start, one cycle, an expectation	44
20.3	Where independence now lives	45
21.	Data: never enumerated	46
21.1	Tokens and terms	46
21.2	Derived arithmetic is enumeration in disguise	46
21.3	The layered value story	46
21.4	When Lean is needed — and when it is not	47
22.	Composition: assume-guarantee	48
23.	Names	48
24.	The certificate: what actually runs	49
24.1	The strong half, and the second configuration	50
25.	Gated datapaths: the <code>opaque_datapath.</code> directive	51
26.	The performance budget: a rule, not a hope	52
27.	Generated designs	53
Part IV — Practice		54
27.1	What the printed RTL must look like	54
27.3	A regular relation is a lane, and a grid has two axes	55
28.	What an entry is	57
29.	The no-regress chapter	58
30.	What carries over from version 1 unchanged	60
31.	Status, migration, and holds	60
31.1	What is machinery today, and what is still owed	60
32.	The ladder: every artifact is a step, and a person approves each one	61
33.	The four semantic decisions	62
33.1	What an object is: the parking space, not the car	62
33.2	Quantifier scope: say it, do not imply it	63
33.3	Correspondence: the courier labels the parcel	65
33.4	Reset scope: the fire-drill rule that cancelled the fire-drill rule	66
33.5	What the four have in common	67
34.	Proving for every size: the depth question	68
34.1	The shortcut that is not allowed	68
34.2	The properties divide in three, and only one third is hard	68
34.3	The window certificate: one slot, promises for the rest	69
34.4	The fold promises are arithmetic, so they go to Lean	69
34.5	What is done and what is owed	70
34.6	N copies of one unit: the stencil, the lift, and the tie	70
35.	The controlled-English surface: the sentence patterns, frozen	71
35.1	Articles and quantity — the words that carry quantifiers	78
35.2	Time — the words that carry instants	79
35.3	State and effect — the verbs	80

35.3a State that is not an object	81
35.4 Requirement and connective words	81
35.5 Choice and fairness — the constructs that are NOT sugar . .	82
35.6 The corpus gate	83
35.7 The pipeline, and the gate on every arrow	83
36. The proof discipline: induction, not simulation, in both engines . .	85

The spec2rtl Methodology

This is the governing document of the spec2rtl route: the method by which an English hardware specification becomes a SystemVerilog design carrying a machine-checked proof that it meets that specification — not for the cycles a testbench happened to run, but for all time.

It is written to be read front to back by someone who knows digital hardware and basic verification but has never seen this route. Motivation comes before mechanism, plain words before terms of art, and a concrete example before the general rule; the precise rules appear along the way, each after the failure it exists to prevent. That register is itself a rule of this repository, not a stylistic preference.

What this document replaces. It merges three documents that had grown apart: the version 2 methodology, the specification layer’s methodology, and the specification language’s grammar. They are one subject — you cannot explain why the language has the shape it does without the proof machinery underneath it, and you cannot use the proof machinery without the language — so they are now one document. The three originals are retired to [archive/spec2rtl2/](#).

How to read it. Part I is the argument: what a specification is, and what it must refuse to become. Part II is the language, and doubles as its reference. Part III is the contract and how it is proven — the machinery that has been built and tested. Part IV is practice: what an entry consists of, what must never regress, and what is still owed. Chapter 0 is the map.

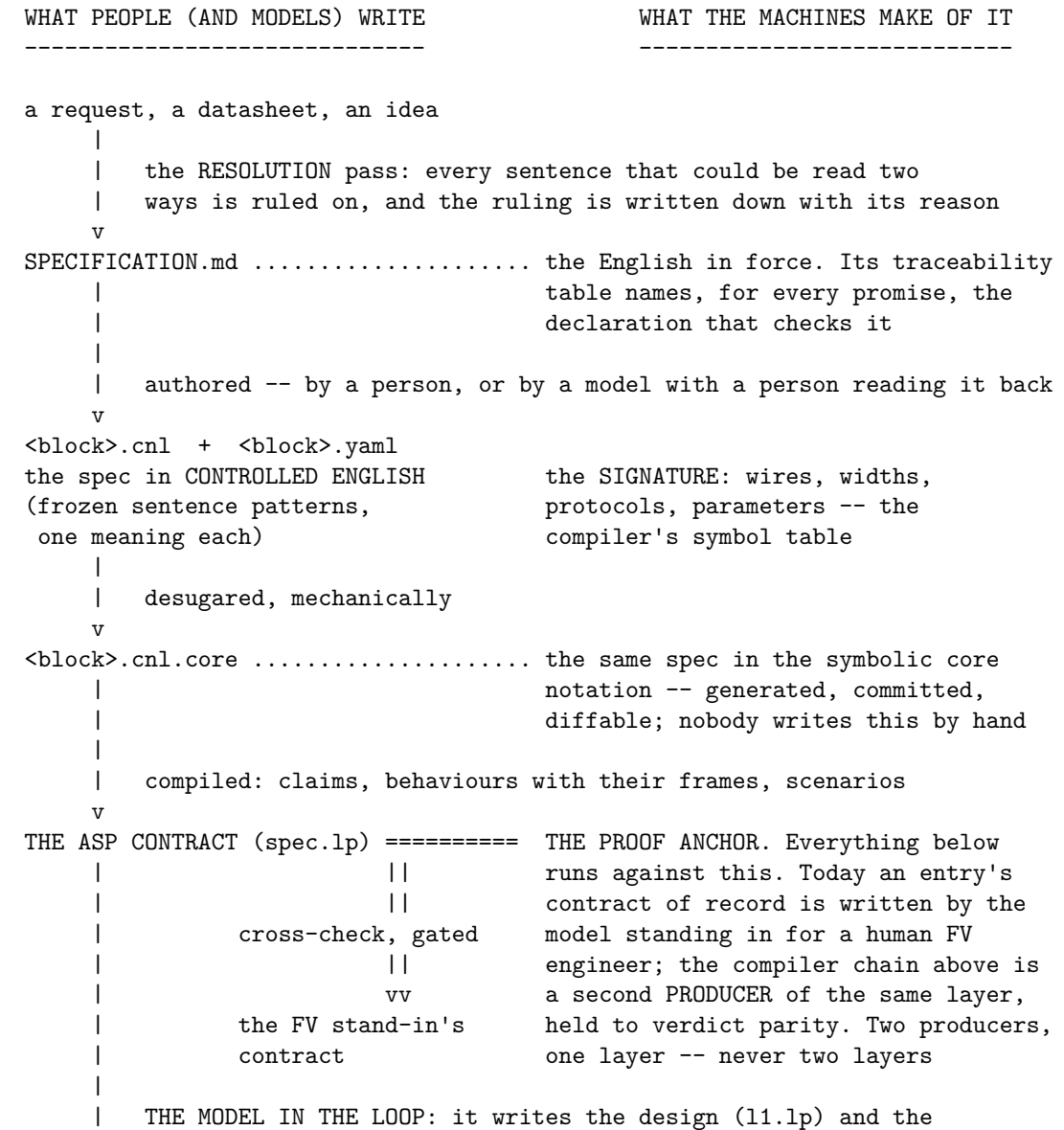
No capability is claimed here until it has its own witness test in `tests/test_aspfir2.py`. Version 1’s operating documents are archived at [archive/spec2rtl_v1/docs/](#).

0. The route, end to end

Imagine you are the architect of a small block — say, the instruction-fetch miss queue this document keeps returning to. You know exactly what it should do: a demand miss allocates an entry or lifts a prefetch that is already chasing the same line; a prefetch reuses an entry when one exists; a second demand for a line somebody is already waiting on stalls the fetch pipe. You could explain all of this to a colleague at a whiteboard in five minutes.

The problem has never been knowing what the block should do. The problem is that between your five minutes at the whiteboard and working silicon sit thousands of decisions, and a testbench only ever checks the handful of cycles somebody thought to run. This route exists to close that gap: a language model writes the RTL — that is the point of the whole exercise — and a proof, not a person's optimism, decides whether it got the block right.

Here is the entire route in one picture. Everything else in this document is a chapter of it.



```

| linkage (l1.inv.lp) against the contract, and each failed
| certificate hands back the exact window where the design
| misbehaves -- a far better instruction than "try again"
v
refine contract l1 --induct K
|
| the CERTIFICATE \--> LEAN (lib/lean), when needed:
| (clingo): control, . an obligation whose terms differ
| proven for ALL TIME . symbolically is "owed to Lean" --
| . never miscalled a failure -- and
| . discharged structurally there
| . a parametric fact (true at EVERY
| . depth) is proven once and borrowed
| . back as a LICENSED fact, because a
| . grounder must be handed numbers
v
print -> roundtrip --sim auto -> SystemVerilog
                                the printed RTL, translated back by an
                                independent translator and compared
                                against the model cycle by cycle, with
                                a third-party simulator arbitrating

```

Now the same picture as a story, because a diagram tells you the shape and not the reasons.

You write two files, and they are written together. The controlled-English specification says what the block does, in sentences with frozen meanings — *@when a valid demand request R arrives ... @then accept R*. The signature says what wires exist and, crucially, which payloads are opaque tokens and which are honest numbers — that one field is the whole control/data split, decided once instead of remembered at every use. Neither file means anything without the other: the specification names no wires, so the compiler reads both or neither.

The English becomes a core nobody edits. The desugaring is mechanical and the result is committed beside the source, so “what did my sentence actually mean?” is a file you can open and a diff you can review. A sentence outside the frozen patterns is refused by name. This matters because free English is precisely the disease the route exists to cure — a specification language that guesses at prose is ambiguity with better marketing.

Everything converges on the contract, and the contract is permanent. The ASP contract is the route’s fixed point: the certificate proves designs against it, the model iterates against it, composition assumes and discharges it. Only its *production* has a story: today the model writes it, standing in for a human formal-verification engineer; the compiler chain is a second producer of the same layer, validated by holding both producers to the same verdicts on the same designs — including deliberately broken ones. (This paragraph is worded

carefully because the point was once gotten wrong in this repository’s own records — the contract was miscast as a transitional artifact awaiting retirement — and the user corrected it. Producers change; the layer does not.)

The model writes the design, and the certificate judges it. This is where the RTL actually comes from — and deliberately NOT from the specification. If the same compiler emitted both the contract and the RTL, any misreading in its front end would appear identically on both sides and cancel, and the certificate would be comparing a thing with itself. The independence that matters is mechanical, not authorial: the same model may write both the spec and the design, because the check never asks who wrote what — it asks whether any execution of this design can violate any property of this contract, and clingo answers.

Two provers, one split. Control goes to clingo, which proves — for all time, from any reachable state — that the design’s decisions obey the contract. Arithmetic goes to Lean, in exactly two shapes: an obligation whose terms differ symbolically is *owed* there and discharged structurally, and a parametric fact is proven once for every parameter and borrowed back under a named license (`Cam.comparator_is_equality` for the CAM’s gate equality, `Rotation.reaches` for the fair arbiter’s coverage). An entry whose obligations all discharge by identity carries no Lean at all, by rule.

One arrow is unprovable, and it is the first one. No theorem reaches from intent to English, or from English to the controlled surface. That arrow gets a handrail instead: the resolution record, the traceability table read both ways by a gate, scenarios that must stay reachable, and a person on every rung of the ladder. Everything below the surface is machine-checked, and every mechanical arrow has a gate that fails loud (Chapter 35.7 walks them one by one).

Part I — What a specification is

1. The problem, and the tool underneath

Suppose you are handed a paragraph of English: *“a FIFO queue, four entries deep, eight bits wide; push and pop; a full flag, an empty flag; data comes out in the order it went in.”* You must produce RTL, and you must be able to say — with something stronger than a testbench — that the RTL does what the paragraph says.

The route’s answer has three movements. First the English is **resolved** into a precise specification: every ambiguity decided and recorded, every checkable sentence tagged with the name of the rule that will check it. Second, the design is **authored as logic facts** (nets, registers, definitions) in a small language whose meaning is fixed by a rule library, and a **certificate** is computed: a set of automated checks that together establish, by induction, that the design can never violate the specification’s properties. Third, the design is **printed** as Sys-

temVerilog and **round-tripped**: the print is translated back by an independent translator and simulated (by Verilator or Icarus, whichever the machine has – the tool prefers Verilator, and Section 27.5 says how a 2-state simulator is kept honest), and all three readings — the authored logic, the translated print, the simulator — must agree on every signal at every cycle of a chosen story.

2. The thesis: what the specification is, and what it is not

A specification and an implementation answer two different questions. The specification says **what relationships must hold** — “an accepted word is delivered exactly once, unchanged”; “a blocked output holds its data steady.” The RTL says **how hardware realizes those relationships** — which registers exist, which muxes select, which enable gates which flop. If writing the specification forces you to name every register, temporary, and priority-encoder stage, the specification has failed: it has merely restated the RTL in a different syntax, and proving the two agree proves only that you transcribed carefully.

Two consequences of this thesis, both easy to get wrong:

Formal detail is not implementation detail. The English sentence “hold a valid output stable under backpressure” expands formally into two precise conditions (valid stays up; data stays put). The formal version is *longer* — because it removes ambiguity — but it is not *lower-level*: it still says nothing about whether the hardware uses a flop enable, a skid buffer, or a retimed pipeline. Length is not the measure of abstraction.

The real measure is implementation freedom. A specification is genuinely above the RTL exactly when *more than one substantially different implementation satisfies it*. This is not an aspiration in this project — it has been measured twice here: the pipelined APB bridge has one English specification and **two proven, structurally different designs**; the FIFO’s certificate once accepted an accidentally different implementation, correctly. The DSL layer makes the criterion operational: an entry may close with a **freedom witness** — a deliberately different second realization certifying unchanged — or, at minimum, an explicit list of the freedoms the contract leaves open (which slot allocates, what order a drain takes, how a comparator is built), each exercised by a scenario.

Structure is allowed in the specification exactly when it is semantically required. “Latency is three cycles when unstalled” is a requirement; “the crossing uses a two-flop synchronizer and a toggle protocol” can be a requirement; “the arbiter is a prefix chain” almost never is. The rule is not *never mention structure*; it is *mention only structure someone outside the block could observe or has demanded*. Every structural mention gets a resolution entry saying why it is required.

The ladder, then, has three rungs, and each rung is a real artifact:

Requirement	"Every accepted word is delivered exactly once."
↓	
Semantic contract the ASP	windows, events, causal promises, invariants -
↓	
RTL	registers, muxes, enables - one of many designs that refine it

One more framing the position paper gets right: assertion languages bolted onto RTL (generated SVA with shadow queues, scoreboards, transaction IDs) tend to become a *second state machine* whose agreement with the first is the new hard problem. In this route the semantic contract is the ground truth itself; a monitor language, if ever needed for industrial interchange, is a *backend* generated from the contract — never the primary specification.

3. The causal layer, and how it compiles — forms, not a compiler

The position paper’s user-facing ideal is the causal rule: $P \rightarrow Q$. “Send and ready $\rightarrow$ accept the word.” “Valid and not ready $\rightarrow$ next data equals data.” This is the right *source* register for a human: a domain expert can audit causality who could never audit a rule soup.

The decision here is **how** that layer exists — and it is deliberately the middle of three options:

- Not a mere *style guide* (that would leave the ideal unfulfilled), and
- not a *compiler* whose output nobody reads (a miscompiled specification proven faithfully is a **false certificate wearing a valid proof** — the single worst failure class this project knows), but
- **causal FORMS with committed expansions**: a small library of named templates — *handshake-accept*, *hold-under-backpressure*, *exactly-once-with-representative*, *within-N-delivery*, *work-conserving-service* — each of which expands **textually** into window-vocabulary promises. The expansion is written into the contract file, readable, and reviewed once per form. The rule that keeps the trust story intact:

**The contract a human reads is, literally, the contract
the machine proves.** A form is a scribe, never an oracle.

A form demands state; it never invents it. Some sentences need memory: “delivered exactly once” has to remember whether a delivery has already happened. The machinery here never invents that memory for itself. Instead, the form turns the need into an obligation on the DESIGN — expose a signal that answers the question, carry a small representative register, or accept a stated bound. The specification’s half of that bargain is a declared **window**: a name, a domain, and a prose meaning, such as **delivered(E)** — “entry E’s word has

gone out.” The design’s half is one **linkage** line that mounts the window on a flop the design already has:

```
delivered(E, T) :- val(sent_flag(E), 1, T).
```

So the window is a *derived view* of the design’s own state — a pane of glass over an existing register, never a copy kept beside it. And the demand is part of the specification’s meaning: declaring the window vocabulary is exactly what constrains the family of acceptable implementations, said out loud instead of implied.

The seam warning — where races live. Take two causal sentences from the miss queue: “when a demand request arrives and a matching entry exists, the request joins that entry,” and “when the memory answer for a line arrives, that entry delivers its data and is freed.” Each is perfectly clear on its own. Now let both fire in the same cycle: a request joins an entry in the very cycle that entry’s answer lands. Does the joining request get the data, or did it arrive one instant too late? Neither sentence says — the question falls *between* them. That meeting point is what this document calls a **seam**: an instant two causal sentences share without either one owning it. Every race this route’s entries have found lived at a seam — the miss queue’s same-cycle fill, the landed-line episode rule, the CDC handshake’s stale-pipe state. Causal forms make the easy majority of a specification frictionless; they must never be allowed to make the seams invisible. Therefore: the adversarial-misreading pass and the resolution record happen **at the causal level too**, every form documents its boundary instants, and a form that would silently default a same-cycle case is refused.

4. Semantic memory: the representation policy

“Every accepted transaction is later delivered” relates two times, so *some* memory is mathematically necessary. The policy for what that memory becomes — decided here, once, so no entry re-litigates it:

1. **Token identity, when the thing remembered is data.** An address, a word, a tag is an opaque token; “the delivered word is the accepted word” is *term identity* — free, exact, never enumerated.
2. **A structural representative, when the thing remembered is a class.** When the specification wants to quantify over an equivalence class — “the line”, “the group”, “the episode” — the class gets a concrete name the hardware carries: an owner’s index, a representative pointer. This is the R9 lesson from the parent’s miss queue, and it is the difference between a proof and a wall: the *relational* spelling (“same line” = address equality between slots) put a quadratic, closure-coupled choice space into every induction start state; the *structural* spelling (a log -depth register naming the owner) made the same fact an index comparison. The user speaks in semantic objects; the form assigns the representative; the window mounts

it.

3. **A bounded window, when the thing remembered is elapsed obligation.** “Delivered within N” is finite safety — cheap and first-class. Bare, unbounded **eventually** is **refused with advice**: choose a bound, a work-conserving spelling (“while an obligation is unmet, the server is serving”), or an explicit ranking argument — the three honest shapes of liveness this route can actually discharge. Silent weakening of **eventually** to “reachable” is forbidden.

And one primitive that looks innocent and is not: **transaction identity T**. Unbounded T-indexed state is where every grounding wall in the route’s history lived. A form that uses T must say which of the three representations above T compiles to — in the expanded text, where the reader can see it.



Part II — The language

The three chapters above argue what a specification must be. This part is the language that makes it writable: its files, its lexical conventions, its complete grammar, the meaning of every construct, and how each expands into the logic the machine checks.

One fact about authorship first, so the rest of this part reads correctly. **Nobody hand-writes the symbolic notation documented here.** What a person writes is the controlled-English surface of Chapter 35 (the `.cn1` file) beside the signature; the desugarer turns those sentences into this notation — the **core**, written beside the source as `<block>.cn1.core`, committed, and never hand-edited. This part is therefore the reference for *reading* a core (which every review does), for what each construct *means*, and for what the compiler accepts — the meaning lives at this level, and the English patterns of Chapter 35 are defined by what they desugar into here.

It is written to be read once end to end; Chapters 9 to 13 are the reference you will return to. The shortest description of the language is that **it is SystemVerilog Assertion notation lifted from wires to architectural objects** — an RTL engineer already knows `|->`, `|=>`, `##N`, `$stable` and `disable iff`, and they mean here what they mean there. What is added is what SVA cannot say without building a scoreboard: transactions and fetches as first-class objects, relations between them, quantification, and counting.

5. What the language is for, and what it refuses to be

A specification says **what relationships must hold**; an implementation says **how hardware realizes them**. If writing the specification forces the author to name registers, muxes, enables, and pipeline stages, the specification has failed

— it is the implementation transcribed, and proving the two agree proves only that the transcription was careful.

The language therefore has no notion of a register, a wire, an enable, or a clock edge in its behavioural layer. Its nouns are **interfaces, transactions, events, semantic state, and relations**. Its verbs are what those nouns do. The one place signals appear is the signature file, where the interface is declared once.

Three design commitments follow, and every rule in this document serves them:

1. **The author writes cause and effect.** The fundamental form is a trigger and its consequence. A domain expert can audit causality; nobody can audit a rule soup.
2. **Behaviour and property are separate constructs.** Behaviour defines how the abstract machine operates; a property states what must be true of every execution of it. Keeping them apart makes one question askable *before any RTL exists* — does the abstract machine satisfy its own claims? — which is where specification defects are cheapest to find.
3. **Nothing is invented behind the author's back.** Semantic memory is demanded, not synthesized: a specification that needs to remember something says so, in its own vocabulary, and the obligation to represent it lands on the implementation.

5.1 The constructs at a glance

The whole language is seven declarations and a handful of qualifiers used inside them. Nothing else exists, and each name says plainly what it is for.

Declarations — the top level of a specification.

the form	it is for
@assume	what the world is granted to do — given, never checked
@index	names an index domain, so a block can have N of something
@state	what the block remembers, in its own terms
@define	gives a concept a name: an event, a condition, a relation, or a value
@behavior	how the machine operates — a cause and its effects
@property	a claim about the machine, checked against it
@scenario	a situation that must stay reachable — the anti-vacuity check

Operators borrowed from SVA — an RTL engineer learns nothing

new here.

the form	it means
$P \ \backslash -> \ Q$	at every cycle P holds, Q holds <i>that same cycle</i>
$P \ \backslash => \ Q$	at every cycle P holds, Q holds <i>the next cycle</i>
$\#\#N \cdot \#\#[a:b]$	N cycles later $\cdot$ somewhere in that window
$s_eventually(Q)$	Q at some later cycle — reduced before it is checked (§11)
$Q \text{ until } R \cdot Q \text{ until_with } R$	Q up to R $\cdot$ up to and including it
$\$stable(x) \cdot \$rose(x) \cdot \$fell(x)$	unchanged $\cdot$ went $0 \rightarrow 1$ $\cdot$ went $1 \rightarrow 0$
$disable \text{ iff } (e)$	the file's claims are void while e holds — normally reset
$\&\& \cdot \backslash \backslash \cdot !$	and $\cdot$ or $\cdot$ not

Operators the language adds, because SVA cannot say them without a scoreboard.

the form	it means
$->$	<i>constructs</i> the machine — only inside @behavior , where $\backslash ->$ is not allowed
$\text{some } X \text{ where } P: \dots \cdot \text{each } X \text{ where } P: \dots$	quantify over transactions or entries
$\text{exactly}(N, \dots) \cdot \text{atMost}(N, \dots) \cdot \text{atLeast}(N, \dots)$	count what satisfies something
$\text{create } \langle \text{kind} \rangle \ X: \dots$	bring a new semantic object into being
$A \ @\text{before } B \cdot @\text{after} \cdot @\text{sameCycle}$	order two events without building a sequence

Two of these carry the weight of the whole design. **@behavior** defines the machine; **@property** makes a claim about it — keeping those apart is what lets a specification be checked against itself before any hardware exists, and it is why the arrows differ: $->$ constructs, $|->$ observes. And **create plus quantification** is the whole reason this is not simply SVA: a fetch is a semantic object that need not exist in any register, and SVA can only quantify over things that do.

5.2 The whole language in one small block

Before any grammar, here is a complete specification of the smallest interesting block there is — a **one-word buffer**. It accepts a word when it is empty,

presents that word until someone takes it, and is then empty again. Every construct of the language appears here, and nothing else is needed. It is shown at the core level — this is the generated file a reviewer reads, not something anyone typed:

```
# oneWordBuffer.cnl.core

disable iff (!resetN)

@assume senderHolds
  input.valid(W) && !accepted(W)
  | => input.valid(W)

@state holding : transaction of input
  meaning: which accepted word the buffer is holding, if any

@define isEmpty
  kind: condition
  meaning: the buffer is holding nothing
  holds when nothing is held

@behavior captureWord
  input.valid(W) && isEmpty
  -> accept W
  ##1 hold W

@behavior presentWord
  holding W
  -> drive output with W.data

@behavior releaseWord
  output.taken
  -> ##1 hold nothing

@behavior assertReady
  isEmpty
  -> ready on input

@property nothingIsLost
  accepted(W) |-> s_eventually(output.taken(W))

@property outputStableUntilTaken
  output.valid && !output.taken
  | => output.valid && $stable(output.data)

@scenario fillsThenDrains
```

```
isEmpty && input.valid(W)
|=> output.valid
```

Read it as five kinds of sentence. The **assumption** is what the world owes the block: a sender holding a word keeps holding it until the block takes it. The **state** is what the block remembers, in its own terms — that it is occupied, and what it holds; not a register, not a flop, just the two facts the specification needs. The **definition** gives a name to a concept the rest of the file uses. The four **behaviours** are the machine, and together they are the buffer’s whole life: **captureWord** takes a valid word in; **presentWord** drives that word onto the output for as long as it is held; **releaseWord** lets it go once the receiver takes it; **assertReady** tells the sender there is room. Each is a cause and its effects, and with the frame rule of section 12 they say completely how the block operates. The two **properties** are claims *about* that machine, checked against it — and note that **nothingIsLost** is not a behaviour: no behaviour says a word will emerge, and the property is what forces the machine’s four rules to add up to that promise. It is also where storing an *identity* pays: because **held** remembers **which** accepted word it is rather than a copy of the bits, the property can name that same **W** on both sides of the sentence. The **scenario** is the anti-vacuity check: if the buffer could never fill and drain, everything above would hold for the worst reason.

Two things to notice about the notation. **The arrows differ by construct:** $\rightarrow$ inside a behaviour *constructs* the machine, while $\mid\rightarrow$ and $\mid\Rightarrow$ inside a claim *observe* it. They are one character apart on purpose — the construct header already tells you which you are reading, and the arrow reminds you. And **disable iff (!resetN) at the top does once** what would otherwise be a reset guard on every line, which is the same job SVA gives it.

5.3 Prior art this language borrows from, deliberately

- **EARS** (Easy Approach to Requirements Syntax) supplies the sentence shape and, more importantly, the discipline of separating a state you are *in* from an event that *happens*. The keywords are gone in favour of SVA’s arrows, but the distinction survives in the kind system (§10.2), where it is still checked — see §16 for what that trade costs.
- **FRETISH** (NASA’s FRET) supplies the field structure — a requirement is a scope, a condition, a component, a timing, and a response — and the discipline of pinning *timing* as a first-class field rather than leaving it implicit.
- **The Dwyer–Avrunin–Corbett pattern system** supplies the semantic core. From 555 real specifications they distilled five patterns (absence, existence, universality, response, precedence) under five scopes (globally, before, after, between, until), covering 92% of what people actually write. The property layer of this language is that pattern set, given a surface syntax.
- **Event-B** supplies the refinement discipline: an abstract machine of events

with guards and actions, refined by a concrete one, related by a gluing invariant that generates proof obligations.

6. The two files

A block is specified in two authored files, and the tool derives a third.

file	contains	its role
<code><block>.yaml</code>	interfaces, parameters, clock and reset	the signature — what wires exist, and each one’s protocol role. Authored.
<code><block>.cnl</code>	sentences in the controlled English of Chapter 35	the specification — what is granted, what the machine is, and what is claimed of it. Authored.
<code><block>.cnl.core</code>	<code>@assume</code> , <code>@state</code> , <code>@define</code> , <code>@behavior</code> , <code>@property</code> , <code>@scenario</code>	the same specification in the core notation this part documents. Generated by the desugarer, committed beside its source, never hand-edited.

The `.cnl` is the formal counterpart of `SPECIFICATION.md`: the same word in two registers, English prose in one and checkable sentences in the other — and the core is what those sentences *mean*, as a diffable artifact. When an architecture is chosen, its realization is not written in this language at all: the design is authored as logic facts (`11.lp`), and its linkage (`11.inv.lp`) mounts the specification’s declared windows on that design’s own registers — Chapter 18 is that story.

The role of a construct is carried by its keyword, not by its file. `@behavior` defines the machine, `@property` claims something about it, `@assume` grants something about the world — and the compiler reads that from the word. This matters more than a file boundary would: a keyword survives copy-paste, a filename does not. The hazard the distinction exists to prevent is real and worth naming: if a claim were ever absorbed into the machine’s *definition*, it would become true by construction, and the proof that reported it would be a tautology. The keyword is what makes that impossible.

The three populations are reported separately. Every run prints how many behaviours define the machine, how many properties are checked, and

how many assumptions are granted, with each named. The split therefore stays visible where it matters — when you are reading a verdict and need to know which sentences were *proven* and which were *given*.

A conventional order within the file, which the linter warns about rather than enforces, because it is the order a person reads in:

1. **@assume** — what the world is granted to do, so the reader knows the context;
2. **@state** and **@define** — the vocabulary this specification speaks;
3. **@behavior** — how the machine operates;
4. **@property** and **@scenario** — what must never happen, and what must remain possible.

Nothing else is a file of this language. English prose lives in `SPECIFICATION.md` and is the statement of what is required; everything generated — the core and the `<block>*.lp` logic below it — is committed and never hand-edited.

7. Lexical conventions

begins a comment, and ## is the delay operator. The two collide, and a lexer that splits on the first # truncates every timed claim in a file at its delay. That is not a hypothetical: it happened while the checker was being built, and the symptom was not a parse error but a check that never fired, because the part of the claim it was about had been cut off before it ever saw it. A comment therefore begins at a # that is not immediately followed by another #.

- **Names** are `lowerCamelCase` and are full words: `capacityAvailable`, not `capAv`. An initial capital is reserved for **instance variables** (`R`, `P`, `E`, `F`), which is also what the target logic requires.
- **Keywords** begin with `@`. No other token begins with `@`.
- **A name says what a thing is, not when it happens.** A behaviour is named for the operation it performs — `captureWord`, `startFetch`, `deliverResponse` — never for its own guard. `offerWhileOccupied` states the guard twice, once in the name and once on the line below it, and reads worse than either alone. Nor is `offerWord` right: *offer* is already this language's word for an input being presented **to** the block, so it reads backwards on an output. It is `presentWord`. A property is named for the claim it makes (`nothingIsLost`, `noDoubleDelivery`), a definition for the concept it introduces (`isEmpty`, `capacityAvailable`), and a scenario for the story it tells (`fillsThenDrains`).
- **Backticks** quote a name drawn from the signature file or from a `@define`, so a reader can see at a glance which words are load-bearing: ``request`` is valid with `R`.
- **Indentation** is significant: a construct's body is indented under its header, and continuation lines are indented further. Two spaces per level.

- **Comments** begin with **#** and run to end of line.
 - **Numbers** are decimal; a parameter name may be used wherever a number may (**@within depth cycles**).
 - A **cycle** is the unit of time. The language has no sub-cycle notion, no clock edges, and no delays other than whole cycles.
-

8. The signature file

The signature says what wires exist. It is built on one rule: **show what a reader needs to see, and write nothing that can be derived.**

What is irreducible: each interface's name; whether the block **receives** or **sends** on it; its payload fields and their types; and the **pin names**, which are there because they are the contract with the RTL and real pin lists do not always follow a convention.

What is derived, and therefore never written: the **protocol** — a **valid** and a **ready** make a ready/valid handshake, a **valid** alone makes a stream that cannot be refused — and **every pin's direction**, which follows from the interface: on one the block *receives*, the **valid** and the payload arrive and the **ready** departs. Writing a direction per pin states the same fact once per wire.

```
metadata:
  block_name: "<name>"
  version: "0.1.0"
  status: "Draft"
  description: "<one line>"

parameters:                                # usable anywhere a number is allowed
- name: "<name>"
  type: "integer"
  default: <number>
  description: "<what it means -- and the DEFAULT is the built entry's configuration>"

clocks_and_resets:
- name: "<signal>"
  type: "clock"
  edge: "posedge"                          # posedge / negedge
  description: "<...>"
- name: "<signal>"
  type: "reset"
  polarity: "active_low"                    # active_low / active_high -- THE SPELLING IS EXACT
  synchronous: "yes"                       # yes / no / unspecified
  discipline: "once"                        # once / recurring
```

```

    description: "<...>"

interfaces:
- name: "<name>"
  protocol: "readyValid"      # readyValid / validOnly
  side: "receives"           # receives / sends
  description: "<...>"

ports:
- name: "<pin>"
  interface: "<name>"        # or "-" for a wire that belongs to no interface
  direction: "input"         # input / output
  width: 1                    # a positive number, or a parameter's name
  elements: <n>               # ONLY for an array port: how many, `width` being one
  clock: "<signal>"
  role: "valid"               # valid / ready / opaque / numeric / level
  active: "high"              # high / low, for a control wire read the other way up
  description: "<...>"

```

Every enumerated value above is checked, and the spelling is exact. That sentence is here because this block once showed `activeLow` while the tool read `active_low`, so a reader following this page wrote a polarity the tool did not recognise — and an unrecognised polarity was taken as active-high. The reset's sense decides which way `disable iff` runs for every monitor in the contract, so the result was not a refusal but a *different specification*, with each claim judged exactly where it should have been silenced. A wrong `role` is slow; a wrong polarity certifies.

8.0a A port's shape, and what a subscript means

A port carries a width; a port that is an **array** carries two facts — how wide one element is, and how many there are — and the pair is not decoration. It decides what a subscript *means*:

the port	<code>data[J]</code> is	it lowers to
a flat vector (<code>width</code> alone)	bit <code>J</code>	a boundary , <code>bit(V, J)</code> — a decision computed from the value
an array (<code>width + elements</code>)	element <code>J</code>	an addressed read, <code>val(data(J), V, T)</code>

The asymmetry is worth understanding, because it is the difference between a claim that scales and one that does not. An **element** is simply a value at an address: an opaque one compares through the equality theory as a token, so a

claim over an array of 32-bit words costs one comparison per position and never enumerates a word. A **bit** is a decision *computed from* a value, which is exactly what a boundary is for — declare the question, read the answer as one free bit, and the word behind it stays unenumerated.

Both readings are available in one specification; what settles which you get is the declaration, never inference. That rule was learned by breaking it: allowing a port subscript at all was motivated by a flat byte, so for a day *every* port subscript was a bit select, and a claim about sixteen words would have compiled silently into a claim about sixteen bits of the first one.

Filled in, that is the entire signature of the one-word buffer of section 5.2:

```
metadata:
  block_name: "oneWordBuffer"
  version: "0.1.0"
  status: "Draft"
  description: "a single-entry buffer with back-pressure"

parameters:
- name: "dataWidth"
  type: "integer"
  default: 32
  description: "width of the word held"

clocks_and_resets:
- name: "clk"
  type: "clock"
  edge: "posedge"
  description: "the edge every other signal is sampled on"
- name: "resetN"
  type: "reset"
  polarity: "active_low"
  synchronous: "yes"
  discipline: "once"
  description: "asserted before cycle 1 and never again"

interfaces:
- name: "input"
  protocol: "readyValid"
  side: "receives"
  description: "the word arriving"
- name: "output"
  protocol: "readyValid"
  side: "sends"
```

```

        description: "the word leaving"

ports:
- name: "inputValid"
  interface: "input"
  direction: "input"
  width: 1
  clock: "clk"
  role: "valid"
  description: "the sender has a word"
- name: "inputReady"
  interface: "input"
  direction: "output"
  width: 1
  clock: "clk"
  role: "ready"
  description: "the buffer can take it"
- name: "inputData"
  interface: "input"
  direction: "input"
  width: "dataWidth"
  clock: "clk"
  role: "opaque"
  description: "the word itself -- routed, never computed with"
- name: "outputValid"
  interface: "output"
  direction: "output"
  width: 1
  clock: "clk"
  role: "valid"
  description: "the buffer has a word to give"
- name: "outputReady"
  interface: "output"
  direction: "input"
  width: 1
  clock: "clk"
  role: "ready"
  description: "the consumer can take it"
- name: "outputData"
  interface: "output"
  direction: "output"
  width: "dataWidth"
  clock: "clk"
  role: "opaque"
  description: "the word held"

```

Six wires, each named once, each with its job — and a reader can see the block’s whole boundary without computing anything. Two facts they will want are already there without being stated: both interfaces are ready/valid (each has a `valid` and a `ready`), and `inputReady` is an output while `inputValid` is an input, because the block *receives* on `input` — and the checker enforces exactly that, so a direction written the other way round is refused rather than believed.

It is more verbose than a sketch would be, and that is the trade this format makes on purpose: every fact is written once, in one place, where a check can reach it.

The roles.

role	what the pin is
<code>clock</code>	the edge every other signal is sampled on
<code>reset</code>	with its polarity — <code>active_low</code> or <code>active_high</code> , and the spelling is exact — and its discipline: once for the standard convention that reset is asserted before cycle 1 and never again, recurring to allow re-assertion at a cost the author accepts
<code>valid</code>	the sender has a transaction on this interface
<code>ready</code>	the receiver can take it this cycle
<code>opaque(width)</code>	payload the block routes but never computes with — an address, a tag, a line of data. It may be compared for equality and copied; never added, indexed, or decoded. This is what keeps a 128-bit payload from costing the proof anything, and a specification that tries to compute with one is refused.
<code>numeric(width)</code>	payload the block genuinely computes with, at the cost that brings

If the pins are already called `req_vld` and `req_rdy`, those are simply the names on the left. Nothing in the specification ever mentions a pin: it speaks of the interface **request**, and the signature is where the two meet.

Protocol roles and the verbs they license. This table is the whole reason the specification can avoid signals:

role	direction	when the world acts (a trigger)	what the block does (an effect)
readyValid	in	<code>`x` is valid with W (a condition)</code>	<code>accept W · refuse W · be ready on `x`</code>
readyValid	out	<code>`x` is taken (an event)</code>	<code>drive `x` with <expr></code>
validOnly	in	<code>`x` arrives with F (an event)</code>	— (nothing to do; it cannot be refused)
validOnly	out	—	<code>send on `x` with <expr></code>
level	either	<code>`x` is high · `x` is low</code>	<code>hold `x` high · hold `x` low</code>

Two rules of voice, and they are the reason this table has two columns rather than one:

A trigger names the interface and the transaction. ``input` is valid with W`, ``fill` arrives with F`, ``output` is taken`. Two earlier spellings were tried and are worth recording as what not to do. “*input is offered*” hides the actor — offered by whom, to whom? — and names no transaction, so there is no word to refer to in the next line. “*input offers W*” fixes the second fault but not the first: *offer* is an action, and an interface is not the one doing it.

valid is the right word because it is the wire’s own. The signature file says `valid: inputValid`, so a reader traces the term straight to the signal with nothing to learn. And *valid* describes a **state** rather than an action, so it needs no actor at all — which is why the earlier trouble disappears rather than being papered over.

Conditions and events are distinguished by which is which. A ready/valid input *is valid* for as long as the sender holds it, so it is a **condition**, and a claim about it holds at every cycle it is true. A fill *arrives* at one cycle, and a transfer *is taken* at one cycle, so those are **events**. The table marks each, and section 9’s kind discipline enforces it.

An effect is an instruction. What the block does is written as a command — `accept W`, `drive `output` with W.data`, `hold W`, `release` — because a behaviour is telling the machine what to do, and a command says who acts without having to name them. The passive “*is accepted*” survives only as an **observation**, for a later sentence that needs to speak about a completed transfer: `accepted(W)`.

Note what this changes beyond readability: acceptance becomes something the block **does**, not something that merely happens to it. `requestReady` is then a

consequence of when the block chooses to accept, which is the honest account of a ready/valid input.

9. The core notation: complete grammar

Written in EBNF. **NAME** is a lowerCamelCase identifier, **VAR** an initial-capital one, **NUM** a number or parameter name, **TEXT** a free-English line (documentation only).

```
specFile      ::= [ disableIff ] { declaration }
disableIff    ::= "disable" "iff" "(" expr ")"
```

```
# The grammar below IS lib/dsl/grammar.ebnf, the language's single source, rendered here
# mechanically (`python -m sv2asp.aspfirst2.dsl.grammar --write`). The PARSEr is built by
# lark from a translation of the same file performed at import (dsl/ebnf.py), so both
# consumers are mechanical and the source stays in the notation people read. Editing this
# block by hand is pointless -- the drift gate reverts to the file's truth.
```

```
expr          ::= conj { "||" conj }
conj          ::= candq | cmp { "&&" cmp } [ "&&" candq ]
candq         ::= quantifier | notq
notq          ::= "!" quantifier
cmp           ::= arith [ CMPOP arith ]
arith         ::= term { ADDOP term }
term          ::= unary { MULOP unary }
unary         ::= notx | delay | postfix
notx          ::= "!" unary
delay         ::= "##" INT unary | "##" "[" INT ":" bound "]" unary
bound         ::= INT | KIND
postfix       ::= primary { trailer }
trailer       ::= fieldtr | calltr | indextr
fieldtr       ::= "." KIND
calltr        ::= "(" [ expr { "," expr } ] ")"
indextr       ::= "[" expr "]"
primary       ::= parenx | num | var | namek | named
parenx        ::= "(" expr ")"
num           ::= INT
var           ::= VAR
namek         ::= KIND
named         ::= DNAME
```

```
quantifier    ::= QUANT KIND VAR [ wherep ] [ scopep ]
wherep        ::= "where" expr
```

```
scopep      ::= ":" expr
```

```
# ---- terminals -----
```

```
QUANT      ::= "some" | "each"
CMPOP      ::= "==" | "!=" | "<=" | ">=" | "<" | ">"
ADDOP      ::= "+" | "-"
MULOP      ::= "*" | "\"
VAR        ::= /[A-Z]\w*/
KIND       ::= /(?!(:some|each|where)\b)[a-z_]\w*/
DNAME      ::= /\$\w+/
INT        ::= /\d+/
```

```
# ---- declarations: the file-level shapes (the structural pass assembles these) -----
```

```
# declaration ::= assume | indexDecl | stateDecl | define | behavior | property | scenario
# indexDecl  ::= "@index" KIND ":" bound gloss
# stateDecl  ::= "@state" KIND [ "[" KIND "]" ] ":" domain gloss
# domain     ::= "flag" | "counter" "(" INT "." INT ")" | "value" "(" bound ")"
#            | "pointer" "(" bound ")" | "transaction" "of" KIND | "index" "of" KIND
#            | "set" "of" KIND | "enum" "{" KIND { "," KIND } "}"
# define     ::= "@define" KIND [ "(" VAR { "," VAR } ")" ] kindLine [ phrasing ] [ gloss
#            [ "holds when" expr ]
# behavior   ::= "@behavior" KIND { binder } expr "->" construction { construction }
# construction ::= [ delay ] ( command | stateUpdate | creation | destruction | scopedEff )
# creation   ::= "create" KIND VAR [ ":" assignment { "," assignment } ]
# destruction ::= "end" VAR
# command    ::= "accept" VAR | "refuse" VAR | "ready on" KIND
#            | "drive" KIND "with" payload
#            | "send on" KIND [ "as" VAR ] [ "answering" VAR ] "with" payload
#            | "hold" KIND ( "high" | "low" )
# payload    ::= expr | assignment { "," assignment }
# stateUpdate ::= expr | assignment -- a flag SET by mentioning it and CLEARED by
#            negating it (~demanding(E) / ~!demanding(E)), or a value assigned
# scopedEff  ::= binder "(" construction { construction } ")" -- the same effects, per
#            object or per position of a declared domain
# assignment ::= target "=" expr
# target     ::= KIND [ "." KIND | "[" expr "]" ]
#            -- a window, one field of an object, or ONE POSITION of an indexed
#            window. The subscript is READ AT THE EVENT'S INSTANT even though the
#            target is written at the next: which position a write lands on is
#            decided by the index now, not by what the index will hold afterwards
# property   ::= "@property" KIND [ "enable iff" "(" expr ")" ] ( claim | "always" expr )
# assume     ::= "@assume" KIND claim
# scenario   ::= "@scenario" KIND claim
# claim      ::= { binder } expr ( "|->" | "|=>" ) [ delay ] expr
```



```
# binder      ::= QUANT KIND VAR [ "where" expr ]    -- scoped by parens, a colon, or a block
```

Three of these productions were added on 2026-08-31, and each was a genuine hole rather than a convenience, found the first time a specification larger than the buffer was read against the grammar:

- **end — the dual of create.** A language that can create objects and never destroy them cannot describe a queue: entries would accumulate for ever and `liveEntries` could only rise. The omission was invisible in the one-word buffer, which has no objects.
- **payload — a field list on drive and send on.** An interface with more than one data wire needs to say what goes on each; `forward` carries a tag *and* a line.
- **always — an invariant with no antecedent.** “The queue never overfills” has nothing to imply from, and writing a trivial antecedent to satisfy the grammar would have been a worse specification.

Two more followed on 2026-08-31, from the semantic decisions of Chapter 33 — see there for the reasoning, which is the interesting part:

- **scope — a quantifier carries one whenever its variable is used beyond the where clause,** parenthesised inline or as a block of claims. A variable exists only inside its scope, so “are these two claims about the same miss?” is answered by looking at the file rather than by knowing a convention. The block form is the one that matters in practice: the interesting claims about a miss are several, and they are only meaningful about the *same* miss. The scope is **optional**, because a quantifier used as a plain condition — “*no entry holds this line*” — never mentions its variable again, and demanding an empty pair of parentheses there would be ceremony for nothing. Several binders may share one scope, which is what “*no two entries hold the same line*” needs.
- **answering on a send** — the behaviour names the object it serves, so the design labels its own answer and the specification reads that label as a window. Without it, a claim about “the entry this answer belongs to” is leaning on a relation nothing establishes.

Three notes that matter more than the syntax:

-> and |-> are different jobs, not stylistic variants. `->` appears only inside `@behavior`, and its right-hand side *constructs*: it creates objects, establishes relations, updates state, and commands the interfaces. `|->` and `|=>` appear only inside claims, and their right-hand side *asserts*. Using the wrong one is a static error, and the construct header always says which you are in.

Every variable must be bound, by the antecedent or by a quantifier, so each rule is a closed statement rather than an implicit existential.

A quantifier ranges over what exists at the cycle it is evaluated. each entry `E` and some entry `E` mean the entries that *exist now* — not every en-

try that ever existed. The object kind is the domain, so **each entry E where inFlight(E)** reads as “for every entry currently in flight”. This is not a convenience: a quantifier over all objects ever created is unbounded, and a proof cannot reason finitely about it. A specification that needs to speak about objects that have already ended must say so, and that is a history question, answered by the representation policy of §10.1 — a **@state**, a relation that outlives the object, or a bound — never by quantifying over the past.

Every object kind carries a built-in exists(E). This was implicit until 2026-08-31, when writing a contract by hand exposed it: **create** makes an object exist, **end** makes it stop, and every quantifier ranges over the ones that exist right now — so existence is already part of the language’s meaning, and a specification that never names it is relying on something it cannot see. Naming it matters because **it is a demand on the implementation**: the design must show which slots are live, and the linkage mounts that as the window every quantifier stands on. Authors rarely write **exists(E)** — the quantifiers imply it — but it is available for the case where an object is held in hand rather than quantified over, and the compiler must demand its window whether or not it is written.

each and **some** are this language’s `forall` and `exists`; the mathematical symbols are deliberately not accepted, on the same one-spelling-per-concept rule that governs the rest of the notation, and because an engineer reading **each entry E where ...** is reading a sentence rather than a formula.

A phrasing: line is sugar with a canonical form behind it. **@define** may give a template — `“R` joins `E”` — usable wherever the relation is allowed; reports and generated logic always use **name(args)**, so the sugar cannot hide meaning.

10. The constructs, one by one

10.1 @state — semantic state the specification names

```
@state credits : counter(0..maxCredits)
  meaning: link credits currently held
```

State is legitimate when it is **architectural**: something the specification itself names, a consumer can reason about, or an external document defines. A program counter, a credit count, a privilege mode, a FIFO’s contents, a cache’s valid bits — all architectural. A pipeline stage’s valid bit, a mux select, an FSM encoding — not, unless the requirement genuinely demands them.

The test to apply, and it is the same test the route uses everywhere: **does the specification name it, or did you introduce it because of how you imagined building the block?** State of the second kind belongs to the realization, and putting it here is exactly how a specification collapses into RTL.

Store an identity, not a copy. When state exists to remember something that came in through an interface, it should hold **which transaction** it is, not a duplicate of that transaction’s payload:

```
@state held : transaction of input      # good -- an identity
@state held : value(dataWidth)         # worse -- a copy of the word
```

Two things follow from the good form. Properties can speak about the transaction — *this word is delivered exactly once, these two are answered in order* — which is what conservation needs and which a bare copy cannot express, because a copy has forgotten where it came from. And the payload is never represented twice, so there is no second copy to keep in step with the first. The value is read off the identity where it is needed: `held.data`. A `value` or `counter` domain is right when the state is a quantity the block *computes* — a credit count, a program counter — rather than something it *received*.

Indexed state, for blocks that have N of something. Declare the index domain first, so it has a name a reader recognizes, and several arrays can share it:

```
@index slot : depth
  meaning: one storage entry of this realization

@state valid[slot] : flag
  meaning: whether entry I currently holds a request

@state occupant[slot] : transaction of request
  meaning: which request entry I holds, while valid
```

The quantifiers of section 9 then read as English over that domain:

```
@behavior allocate
  accepted(R) && some slot I where !valid[I]
  -> ##1 valid[I] = 1
      ##1 occupant[I] = R
```

Note what the existential is doing: *some* free entry takes the request, and which one is not said. That is implementation freedom expressed exactly where it belongs — in the specification’s own nondeterminism — rather than left to a comment.

Expansion. A `@state` declaration becomes a declared symbol read at every instant, mounted by each implementation’s linkage on whatever really holds it:

```
% @state credits : counter(0..8)
% declared; each realization's linkage defines it, e.g.
%   credits(V, T) :- val(creditCounter, V, T).
```

A pointer(N) domain, and three relations on it. A queue’s whole state is two pointers, and every claim about it is one of three relations: they are equal (empty), they are a lap apart (full), or one has advanced. Written with + and % those would be arithmetic in the step vocabulary, which this route forbids because it enumerates. Written as `next(P)`, `address(P)` and `opposite(P, Q)` they are three named relations on a small domain — the hardware’s own vocabulary, which is what the no-derived-arithmetic rule asks for rather than merely permits.

The domain was added on 2026-08-31, when the FIFO became the second block written in this language and turned out not to be expressible in it. That is the useful kind of finding: the first block had objects and a CAM and no pointers at all, so nothing had yet asked the language for the one piece of vocabulary every queue needs.

10.2 @define — giving a semantic concept a name

```
@define outstanding(R)
  kind: condition
  phrasing: "`R` is outstanding"
  meaning: R has been accepted and has not yet been delivered

@define capacityAvailable
  kind: condition
  meaning: fewer than `depth` requests are outstanding
  holds when atMost(depth - 1, some request R where outstanding(R))
```

A definition with a `holds when` body is **derived** — the compiler expands it. A definition without one is **primitive**: it has a name, a kind, and an English meaning, and behaviours establish it. Primitive relations are how a specification says “these two transactions are related” without saying how anything stores that fact.

kind is one of:

- **event** — occurs at a cycle (an acceptance, an arrival, a delivery). Usable after as an antecedent, in scopes, and in `@before` / `@after`.
- **condition** — holds over cycles (outstanding, capacityAvailable, joinable). Usable as an antecedent, as a conjunct, and on either side of `until`.
- **relation** — relates two or more instances (joins, corresponds, satisfies). Established by behaviours, persists under the frame rule.
- **value** — a derived value (an address, a count).

10.3 @behavior — how the abstract machine operates

```
@behavior startFetch
  accepted(R) && !some fetch E where E.address == R.address && joinable(E)
  -> create fetch F: F.address = R.address
```

```

joins(R, F)
##1 wantsFetch(F)

```

A behaviour is a **cause and its effects**. Together, all behaviours define the abstract machine's transition relation — and, with the frame rule of section 12, they define it completely.

Expansion. Each effect becomes a positive rule whose body is the trigger and its conjuncts:

```

% @behavior mergeRequest
joins(R, E, T) :- accepted(request, R, T), fetchFor(E, A, T), addressOf(R, A, T),
                  joinable(E, T).

```

Timing modifies the head's instant: `none` means the same cycle, `@next` means `T+1`, `@within N cycles` generates the bounded obligation of section 11.

10.4 @property — what must be true of every execution

```

@property noDoubleForward
  each request R
  |-> atMost(1, some forward D where corresponds(D, R))

```

A property is a **claim about the machine**, never part of its definition. Properties expand into the violation form: the specification names each kind of failure, and the checker proves no failure is derivable.

```

% @property noDoubleDelivery
failType(noDoubleDelivery, T) :- corresponds(D1, R, T1), corresponds(D2, R, T2),
                                D1 != D2, T = T2.

```

The author never writes a negation or a `failType` head; the expansion produces both.

10.5 @scenario — what must remain possible

```

@scenario mergeAvoidsSecondFetch
  request.valid(R) && some fetch E where E.address == R.address && joinable(E)
  |=> joins(R, E)

```

A scenario is checked in the direction opposite to a property: the situation must be **reachable**, and the outcome must then be unavoidable from it. A specification whose scenarios cannot be reached is vacuous — every property holds because nothing happens — and this is the check that catches it. A

scenario failure reads “*this situation is impossible*”, which is a different sentence from “*this claim was violated*”, and the report says which.

10.6 @assume — what the environment promises

```
@assume fillsAnswerFetches
  fill.arrives(F)
  |-> some fetch E where E.address == F.address && inFlight(E)
```

An assumption constrains the *world*, not the block. It expands to an integrity constraint: executions violating it are excluded from consideration rather than reported as failures. Every assumption is an obligation on whoever integrates the block, and the generated report lists them, because an assumption nobody validates is a hole in the proof wearing a green tick.

11. Time

SVA’s operators mean here what they mean there, with each boundary pinned because this is where readers and writers most often disagree. Let an event *E* occur at cycle *e*.

the form	it holds when
$P \setminus \rightarrow Q$	at every cycle <i>P</i> holds, <i>Q</i> holds at that same cycle
$P \setminus \Rightarrow Q$	at every cycle <i>P</i> holds, <i>Q</i> holds at cycle + 1
$\#\#N \ Q$	<i>Q</i> at exactly <i>N</i> cycles after the antecedent
$\#\#[a:b] \ Q$	<i>Q</i> at some cycle in that window — a bounded obligation
<i>Q</i> until <i>R</i>	<i>Q</i> at every cycle from now up to but excluding <i>R</i> ’s cycle
<i>Q</i> until_with <i>R</i>	as above, including <i>R</i> ’s cycle
<i>s_eventually</i> (<i>Q</i>)	<i>Q</i> at some later cycle — see below
<i>\$stable</i> (<i>x</i>)	<i>x</i> has the value it had last cycle
<i>\$rose</i> (<i>x</i>) · <i>\$fell</i> (<i>x</i>)	<i>x</i> went 0→1 · 1→0 across the last edge
<i>\$past</i> (<i>x</i>)	<i>x</i> ’s value last cycle — depth 1 only (§16)
<i>A</i> @after <i>B</i>	at cycles <i>t</i> ≥ <i>e</i> — inclusive of <i>B</i> ’s own cycle
<i>A</i> @before <i>B</i>	at cycles <i>t</i> < <i>e</i> — exclusive
<i>A</i> @sameCycle <i>B</i>	both at the same cycle — a coincidence made explicit

the form	it holds when
<code>disable iff (e)</code>	every claim in the file is void at cycles where <code>e</code> holds

Why @before, @after and @sameCycle survive alongside SVA. SVA can express ordering with sequences, but `A ##[1:$] B` combined with `throughout`, `within`, `intersect` and `first_match` becomes unreadable quickly, and a reader must reconstruct the ordering relation from the machinery. `accepted(R) @before fill.arrives(F)` says it directly — and says it about *objects* rather than signal windows, which is what makes it possible at all.

On `s_eventually`. It is legal in a claim and never in a behaviour — a machine definition must say what happens, not that something happens someday.

It is proven by **induction**, like everything else here: assume the claims at the window's start with semantic state and inputs free, take one step, look for a violation. It needs one preparatory move, because induction refutes *reachable bad states* and a bare eventuality has none — every finite prefix in which the event has not yet occurred is perfectly legal. Left as written, the step would succeed trivially and prove nothing, which is exactly the vacuity this route treats as its cardinal failure. So the claim is first **reduced** to a one-step obligation, in one of three ways:

- **a bound** — `##[1:N]` makes the violation a state (an age reaching $N+1$), and the induction proves that bound as its own invariant: assume the age is at most N at T , take one step, show it is still at most N at $T+1$, with the base case grounding it at zero out of reset;
- **a ranking** — a measure that strictly decreases while the obligation is unmet and is bounded below; both halves are one-step, hence inductive;
- **work conservation** — *while any obligation is unmet, the machine is serving one*, checkable at every instant, which with a finite outstanding set gives the same conclusion.

The bound is usually **discovered rather than authored**: try $N = 1, 2, 3 \dots$ and take the first that proves inductively, recording the discovered N in the report as part of what was proven. What the compiler must never do is accept a bare `s_eventually` and check nothing.

Lasso search keeps one job, and it is diagnosis, not proof. When no reduction succeeds, a reachable cycle of the abstract machine in which the promised event never occurs is the concrete starvation trace — the answer to *why*, not merely *whether*. It is affordable here precisely because the behavioural model is small.

12. The frame rule

Nothing changes unless a behaviour causes it. Semantic state keeps its value, and an established relation persists, until some behaviour’s effect changes it. This is not a convenience; it is the completion of the causal rules, and it is what makes a set of `@behavior` declarations a *complete* transition relation rather than a partial one.

Two consequences the author should internalize:

- You never write a “hold” rule. Writing one is a symptom of thinking in states rather than events, and it is how specifications accidentally forbid legal designs. Concretely, in the one-word buffer you do **not** write:

```
@behavior holdWord          # WRONG -- do not write this
  holding W
  -> ##1 hold W
```

It is unnecessary, because the frame already says `held` keeps its value unless a behaviour changes it. Worse, it is *harmful*: it forbids a design that releases a word and accepts a new one on the same edge — occupied before, occupied after, a different word rightly held — which no requirement ever asked to forbid.

- If two behaviours’ effects contradict at the same cycle, that is a **static error**, not a resolution by priority. The compiler reports the pair, and the author decides which trigger to narrow. Priority orderings hide exactly the seams this language exists to expose.

12.0a A window holds ONE value, and that is checked

Before the frame can say a window did not change, something has to say what “the value” of a window *is*. Nothing did.

A window is a derived view of the design, and in practice every window is mounted from exactly one `val/3` atom — which the translator’s schema makes single-valued for each signal each cycle — so the property came for free and was never stated. A linkage that mounts one window from *several* rules or several signals has no such guarantee: a one-hot phase whose two bits are both high derives `phase(idle, T)` and `phase(presenting, T)` at one instant.

The failure is the masking direction, which is why it matters. A claim lowers to `w(V, T)`, `V = x` — it asks whether *some* value of the window is `x`. So a spurious second value does not raise a false alarm; it **satisfies a claim the design violates**. A machine sitting in `idle` with `done` asserted passes “done implies presenting”, because some other value of the window happens to be `presenting`.

So the compiler emits, for every value-carrying window, a monitor:


```
failType(phaseNotSingleValued, T) :- live(T), phase(A, T), phase(B, T), A != B.
```

A monitor, and deliberately not an integrity constraint. The obvious spelling is `:- phase(A,T), phase(B,T), A != B.` — but a constraint *excludes* the multi-valued executions, so a linkage that really is multi-valued makes the program unsatisfiable, and UNSAT is read as “no counterexample”. That is precisely how two translator defects once hid behind a check that passed, and it is why this route requires a conformance check to demand its goals be reachable. A monitor reports the linkage by name instead. And because the induction step assumes the property set over its window, the monitor doubles as the assumption that stops a free start from inventing multi-valued states — so the corresponding property discharges structurally rather than needing a transcription of its own.

The comparison is TERM inequality, not the equality theory, and that is not the opaque-compare rule being broken. The question is not whether two values are equal but whether the window was mounted twice, and two distinct terms mean two distinct `val` facts whatever they denote.

A window the author declares SET-VALUED is legitimately many-valued and is exempt — by its declaration, never by inference, like every other shape decision in this language.

12.1 What the frame is keyed BY

The frame monitor asks the same question of every window — *did this change with no cause that licensed it?* — but it has to ask that question of something, and what varies between windows is the **key**: the thing the answer is indexed by.

the window	its key	the guard that goes with it
a scalar — a phase, a counter	none	the window’s value alone
indexed by a declared <code>@index</code> domain	the position	the position ranges over the declared extent
a field or flag of an object	the object	the object must exist at BOTH instants

The third row is where allocation and death would otherwise be reported as frame violations, which is why it carries the existence guard and the other two do not: a scalar has no lifetime to begin or end, and a position is always there.

A licence is specific to its key. When a behaviour writes one position — “capture the line into the bit the counter points at” — the licence records *which* position, read at the instant the event happened. A frame that licensed every

position instead would emit the same monitor, read correctly to a reviewer, and forbid nothing at all.

Only a window some behaviour WRITES is framed. A window the specification merely reads is a derived view of the design, not state the specification took responsibility for: the miss queue’s `liveEntries` is the count of what currently exists, and a frame on it would forbid the design from changing something no requirement ever claimed to control.

This section exists because the rule was once implemented only for the third row, and the consequence was not a refusal. A block whose state was a phase, a counter and an array compiled cleanly, reported every monitor, and certified green — while nothing in its contract forbade the design from changing a captured bit between the cycle it was captured and the cycle it was presented. See `LEARNINGS.md`, “The frame rule was object-shaped”.

13. Well-formedness — what the compiler checks before anything is proven

Each rule is given with what it rejects, because the refusal teaches faster than the rule.

1. **Declared vocabulary.** Every name resolves to a signature interface, a parameter, an `@index`, a `@state`, or a `@define`. There are no undeclared words. *Rejected:* `-> the buffer is drained` — nothing declares `drained`.
2. **Kind discipline.** Relations appear with their declared arity; `until` and `until_with` take conditions on both sides; `create` is triggered by an event. *Rejected:* `outstanding(R) until fill.arrives(F)` — the right side is an event, and `until` needs a condition to hold up to. 2b. **Arrow discipline.** `->` appears only inside `@behavior`; `|->` and `|=>` only inside claims. *Rejected:* `@property ... -> create fetch F` — a claim constructing the machine.
3. **Binding.** Every variable in an effect or consequent is bound by the trigger or a quantifier. *Rejected:* `fill.arrives(F) |-> joins(R, E)` — neither `R` nor `E` is bound.
4. **Opacity.** An opaque payload is compared or copied, never computed with. *Rejected:* `##1 hold W.data + 1` — arithmetic on an opaque word.
5. **Effect legality.** A `@behavior` effect may constrain outputs, state, and relations. It may not constrain an input — inputs are the environment’s, and constraining one is an `@assume`. *Rejected:* `@behavior senderBehaves ... -> ##1 input.valid(W)` — a behaviour dictating what the sender does; it is not a command, which is the tell. The same sentence is legal as an `@assume`, which is exactly where the one-word buffer puts it.
6. **Purity of properties.** A `@property` may read anything and establish nothing. *Rejected:* `@property ... |-> joins(R, E)` — a claim quietly

creating a relation, so that it holds by construction.

7. **No contradictory effects** at the same cycle (section 12). *Rejected:* one behaviour with `-> ##1 valid[I] = 1` and another with `-> ##1 valid[I] = 0`, both able to fire together. The compiler names the pair; the author narrows a trigger. It never resolves the clash by priority.
8. **Reachability.** Each `@behavior` must be able to fire in some execution, each `@property` must have a satisfiable antecedent, and each `@scenario`'s situation must be reachable. *Rejected:* a behaviour whose trigger contradicts an assumption — it can never fire, so whatever it promised is never delivered, and nothing else would ever have said so.
9. **Index discipline.** A subscript names a declared `@index` domain, and every index variable is bound by a quantifier before it is used. *Rejected:* `-> ##1 valid[I] = 1` with no `some slot I` or `each slot I` binding it — the sentence does not say which entry. 10a. **Scope.** A variable is used only inside its quantifier's scope. Where no scope is written the scope is the **where** clause, so a quantifier used as a plain condition needs nothing further. *Rejected:* `some entry E where inFlight(E) |-> ... E.address ...` — the antecedent reports only *that* such an entry exists; it hands you no entry to carry across the arrow. The compiler names the enclosing scope the author probably meant, which is almost always `each entry E ( ... |-> ... )`. 10b. **Lifetime.** A claim mentioning a bound object at two different instants must say whether it still exists at the later one. *Rejected:* `filled(E) && demanding(E) |-> ##[1:depth] forwarded(E)` — sound only if entries never end, and if they do, the entry that *replaced* this one can discharge its obligation. 10c. **Correspondence.** A `send` whose object any claim refers to must name it with **answering**, and the named window joins the mount manifest. *Rejected:* a claim about “the entry this answer belongs to” where no `send` says which entry that is — the relation would exist only in the prose. 10d. **Never-fired.** Not a parser check but a certificate one, listed here because it is what makes the reset rule safe: every run reports the properties that could not have fired at any instant of it. A property silenced by a `disable iff` announces itself instead of passing quietly.
10. **State discipline.** Every `@state` carries a meaning line, and two questions are asked of each: *does the specification name this, or did you introduce it because of how you imagined building the block?* (section 10.1's architectural test), and *is this remembering an identity, or copying a payload it could look up?* *Rejected on review, not by the parser:* `@state stageTwoValid` in a block whose requirement never mentions pipeline stages.

Checks 10a to 10d exist because of the four semantic decisions of Chapter 33, and each is there for the same reason: the thing it catches **fails silently**. A specification that gets one of them wrong does not break — a sentence in it simply stops meaning anything, and the certificate still reports success.

14. The obligations

The language exists to make four questions askable, in this order:

1. **Is the specification consistent and non-vacuous?** Every behaviour fires somewhere; every property's antecedent is reachable; every @scenario's situation is reachable and its outcome unavoidable from there.
2. **Behaviour |= Property** — does the abstract machine satisfy its own claims? Checked *before any implementation exists*, by the same induction the rest of the route uses: the property set assumed at the window's start, semantic state and inputs free, one step, no violation. Eventualities are reduced first (section 11). This is where starvation, lost transactions, and under-specified arbitration are found — as specification defects, not as RTL bugs.
3. **Realization refines Behaviour** — does the implementation refine the abstract machine? Discharged in the Event-B manner: a mapping from the realization's state to the abstract machine's, with each concrete event refining an abstract one. Properties then transfer for free.
4. **No over-constraint** — does the realization forbid anything the behaviour permits? Refinement is blind to this direction, so it is checked separately: a behaviour-legal trace the realization cannot produce is a defect *in the realization's specification*, not in the design that was rejected.

15. Worked example: the miss queue

The complete specification of the instruction-fetch miss queue lives beside its English in `examples/spec2rtl2/rv_missq/` — `rvMissq.yaml`, `SPECIFICATION.md`, `rvMissq.cnl` with its generated `rvMissq.cnl.core`. Its shape at the core level, abridged to the parts that show what the language is for:

```
disable iff (!resetN)

@assume fillsAnswerFetches
  fill.arrives(F) |-> some entry E where E.address == F.address && inFlight(E)

@define joinable(E)
  kind: condition
  meaning: E exists and its fill has not arrived, this cycle included

@define eligible(E)
  kind: condition
  meaning: E wants its enquiry made, and either it is demanding or no demanding entry waits
  holds when wantsFetch(E)
```

```

    && (demanding(E) || !some entry U where wantsFetch(U) && demanding(U))

@behavior allocateDemand
    request.valid(R) && R.isDemand == 1 && roomForDemand
    && !some entry E where E.address == R.address && joinable(E)
    -> accept R
        create entry N: N.address = R.address, N.tag = R.tag
        ##1 demanding(N) && wantsFetch(N)

@behavior liftPrefetchToDemand
    request.valid(R) && R.isDemand == 1
    && some entry E where E.address == R.address && joinable(E) && !demanding(E)
    -> accept R
        ##1 demanding(E) && E.tag = R.tag

@behavior mergePrefetch
    request.valid(R) && R.isDemand == 0
    && some entry E where E.address == R.address && joinable(E)
    -> accept R

@behavior stallRepeatDemand
    request.valid(R) && R.isDemand == 1
    && some entry E where E.address == R.address && joinable(E) && demanding(E)
    -> hold fetchStall high
        refuse R

@behavior receiveFill
    fill.arrives(F)
    -> each entry E where inFlight(E) && E.address == F.address:
        ##1 !inFlight(E) && filled(E) && lineData(E) = F.data

@property demandFirst
    memoryRequest.valid(A) && some entry U where wantsFetch(U) && demanding(U)
    |-> some entry E where E.address == A && demanding(E)

@property noDemandLost
    demanding(E) |-> s_eventually(forwarded(E) || !demanding(E))

```

Every one of those reads to an RTL engineer as an assertion — and not one could be written in SVA without first materializing `entry`, `demanding` and `lineData` as signals or scoreboard state, which is precisely the contamination this language exists to avoid.

One modelling decision is doing most of the work, and it is worth naming. An entry is a **line being fetched**, not a request. That is what makes merging expressible in a single sentence: a demand arriving for a line already being prefetched does not take an entry — it *lifts* the entry that exists,

which acquires the demand's tag and its priority. A prefetch arriving for a line already demanded takes nothing and changes nothing. And a *second* demand for a line already demanded has nowhere to put its tag, which is exactly why the specification stalls the fetch pipe rather than merging: an entry holds one demand tag, and the alternative would need a list.

The reservation follows from the same fact. It is headroom for *allocation*, not a dedicated place — a lifted entry consumes no free entry at all, so it neither needs nor occupies the reserved one.

16. What is borrowed from SVA, and what deliberately is not

Borrowed: the spelling. `|->`, `|=>`, `##N`, `##[a:b]`, `s_eventually`, `until`, `until_with`, `$stable`, `$rose`, `$fell`, `disable iff`. An engineer who knows SVA reads these correctly on sight, and that is most of the ergonomic argument for the language.

Not borrowed: SystemVerilog's scheduling model. SVA's semantics rest on sampled values in the preponed region, on the active / observed / reactive regions, and on glitch behaviour within a time step. None of that exists here: a cycle is one tick and there is nothing below it, exactly as §7 says. The operators therefore mean what a reader expects *at cycle granularity* and no more. A block that genuinely needs sub-cycle behaviour states it in SPECIFICATION.md as a departure; it cannot be written in this language.

Not borrowed: sequences. `throughout`, `within`, `intersect`, `first_match` and `##[1:$]` chains are absent. Ordering is expressed by `@before`, `@after` and `@sameCycle`, over objects rather than signal windows.

\$past is depth 1 only, and `$stable`, `$rose`, `$fell` are its harmless derivatives. Deep lookback — `$past(x, 8)` — is history reconstruction, and history reconstruction in a monitor is exactly the shadow state machine §5 refuses. Something that must be remembered across many cycles is remembered by a `@state`, a relation, or a bound (§10.1), where it is visible and named.

s_eventually is spelled the same and treated more carefully. SVA tools handle strong eventualities with liveness engines of their own; here it must be reduced before it can be checked at all (§11). The spelling is familiar; the discipline is this route's.

One thing genuinely lost, and worth stating. The earlier notation had two trigger keywords — `@when` for an event, `@while` for a state — which made a common over-constraint impossible to *write*. SVA's arrows do not distinguish them. The mitigation is twofold and, on balance, adequate: the **kind system** (§10.2) still classifies every predicate as an event or a condition and checks the uses that depend on it, and the **frame rule** (§12) is what actually prevented

the bug that distinction guarded against. The gain in familiarity is worth the loss; if experience says otherwise, restoring the pair costs nothing.

And the direction this opens: SVA as a backend. A claim over signals alone — `quietUnderReset`, `outputStableUntilTaken` — compiles almost literally into an SVA assertion, so one source can also drive an RTL check. A claim that quantifies over abstract objects cannot: emitting it as SVA would require materializing those objects as auxiliary state, which is the scoreboard problem this language exists to avoid. The honest split is therefore **signal-level claims to SVA, object-level claims to the proof engine**, with the report saying which of a file’s claims can be exported and which cannot.

17. What is deliberately not in the language

- **No sub-cycle time.** No edges, no delays, no phases. A design needing them states them in `SPECIFICATION.md` as a departure.
 - **No priority resolution between behaviours.** Contradictions are errors, not something to be settled by ordering (section 12).
 - **No arithmetic on opaque values.** If a specification must compute with a payload, the payload is not opaque and the signature file must say so.
 - **No implementation vocabulary.** There is no way to write a register, a mux, a stage, or an enable. That is not an omission to be fixed later; it is the point.
 - **No unbounded @eventually in behaviour.** A machine definition says what happens.
 - **No more than roughly three conjuncts per trigger.** Past that the sentence stops being readable and the requirement wants a table or a state machine instead — a limit the EARS authors document from field experience, and one worth respecting.
-

Part III — The contract, and how it is proven

This part is the machinery: what the contract is written over, how it is proven for all time, how vacuity is caught, what happens to data, how designs compose, and what actually runs when a certificate is produced. Unlike Part II, none of it is prospective — every mechanism here is built, and every claim carries a witness test.

18. The central idea: properties on the outside, linked to the inside

18.1 What the ports can already say

Begin with the FIFO, which is this chapter’s running example. Its interface:

```
push, pop, data_in[8] → [ fifo ] → full, empty, valid_pop, data_out[8]
```

A surprising amount of the specification needs nothing but these ports. “The FIFO never asserts full and empty together” is a sentence about two output wires. “After reset, empty holds until the first push” mentions ports and time, nothing else. In version 2 every such property is written directly over the interface, as a named rule that derives a *bad* atom when — and only when — that particular kind of wrongness occurs:

```
bad(full_and_empty, T) :- val(full, 1, T), val(empty, 1, T).
```

The name matters. A certificate that fails tells you *which* *bad* fired and at what instant, so the name is the first line of the diagnosis. This is the first tenet:

Tenet 1 — Properties live on external symbols, and the interface signals are external symbols. Every property — safety, liveness, function — is stated over the design’s ports directly, plus whatever linked symbols the ports do not expose, with one named *bad* predicate per kind of wrongness. A property that needs only ports needs no other machinery at all.

18.2 What the ports cannot say, and the wrong way to fix it

Now try to state “data comes out in the order it went in.” The ports show single pushes and single pops; the *order* lives in the storage between them. The specification needs to see inside.

Version 1’s instinct — the standard instinct, inherited from simulation-era scoreboards — was to build a **ghost**: a specification-side notebook that watches the ports and keeps its own queue. “On a push, append *data_in* to my list; on a pop, check *data_out* against my head and drop it.” This reads naturally, and it is a trap. The notebook is a second state machine: it must be initialized, its contents must be proven consistent with the design’s in every reachable state, and every one of its captured values is a token the grounder will enumerate combinations of — slot × value × value × control. Measured on a real out-of-order pilot, that bookkeeping ground six hundred thousand candidate tuples and sat at a 900-second limit with **Solving**: 0.00s: all the cost went to checking the copy, none to checking the design.

18.3 Windows: seeing the design's own state, without copying it

Version 2's replacement is the **window**: a derived view of the design's *own* flops, defined at every instant, carrying no state of its own. The specification does not keep a queue; it looks at the queue the design already has.

Concretely, the FIFO specification *declares* three windows as free vocabulary — a name, a domain, a prose meaning:

```
pointer_push(P, T)  -- what the design uses as its push-side pointer, at instant T
pointer_pop(P, T)   -- likewise for the pop side
cell_value(A, V, T) -- what the storage holds at address A
```

and states every internal property in terms of them: the ordering property says *whatever these windows show* must have the first-in-first-out relation. The windows are not yet connected to anything — the specification is written before any design exists.

When a design *is* written, it brings a one-line-per-window adapter, the **linkage**, in its own companion file:

```
pointer_push(P, T) :- val(write_ptr, P, T).
pointer_pop(P, T)  :- val(read_ptr, P, T).
cell_value(A, V, T) :- val(cell(storage, A), V, T).
```

Each rule is a pure derived view: defined at every instant, from the design's registers, with no event, no capture, and no state. A different implementation of the same specification mounts the same windows onto *its* registers with different right-hand sides. The specification is the fixed contract; the linkage is the per-design adapter.

This shape is not a hope; it is the fastest thing in the corpus. The Am2901 bit-slice processor — a 16-register file and a Q register, 2^{68} states — proves inductive in three seconds precisely because its specification carries no notebook at all: the register file is state the datasheet itself names, so the specification simply reads the design's own registers and states the transition relation over them.

Tenet 2 — Symbols are linked, not mirrored. Where a property needs more than the ports, the extra symbols are connected to the RTL by linkage rules driven from the internal flops and memories — a derived view, defined at every instant, carrying no state of its own.

Tenet 2b — The spec declares the windows; each level mounts them. The spec's linked symbols are free vocabulary until a level's own linkage file gives them glass.

Two consequences deserve to be stated rather than discovered. First, the window vocabulary constrains the implementation family: a specification that speaks of pointers fits pointer-disciplined storage, and an implementer who chooses a fundamentally different organization will write a more interesting linkage or renegotiate the specification — either way, visibly. Second, and more dangerous: **an unmounted window makes its monitors silently vacuous**. A monitor over `cell_value` on a level that never defines `cell_value` can never fire, and “never fires” looks exactly like “always satisfied.” The runner must therefore report, per level, which spec symbols are undefined and which monitors are dormant — the *window with no glass* check. (This check is owed as of this writing, recorded with a sabotage test in the same family as version 1’s dark-read guarantee.) A window that is mounted but *lies* is caught behaviourally: the bounded stories compare the ports against the design-plus-linkage at every instant, and a linkage that misdescribes the design contradicts the ports somewhere.

There are exactly two places where an event-driven ghost remains permitted, both because the enumeration wall is a *composition-scale* phenomenon: under a `refmodel` gate (a language mechanism the lint understands and the bounded legs assert as a fact — the base, the scenarios, the delivery obligation — while the step does not; this is how a WIDE expected value is stated as a `model` without grounding it in the step), and inside a **unit’s contract at standalone scale** — a unit’s job-capture ghost proves in seconds standalone and never enters the composed step (Chapter 22).

19. Induction in normal form

19.1 The problem induction solves: you cannot walk there

Suppose you want to check that a full FIFO refuses a push. The obvious plan is to *simulate your way there*: reset the design, push four times, and now examine the full state. On a four-entry FIFO this is four cycles of walking. On a design whose interesting state is a thousand cycles deep — a saturated pipeline, a nearly-wrapped counter, a cache in a particular fill pattern — the walk costs a thousand cycles of solver work *per property per attempt*, and worse: a walk reaches *one* full state, the particular one your input sequence built, while the claim is about *every* full state. Bounded checking from reset inherits both problems: its cost grows with depth, and its coverage ends at the horizon. To prove something for **all time**, walking is the wrong tool entirely.

19.2 The move: abstract the initial state

The escape is to stop reaching states and start *describing* them. Instead of starting the window at reset and walking forward, the step starts the window **in an arbitrary state**: every register and every memory cell is freed to hold any value at the window’s first instant. The full FIFO is no longer four pushes

away — it is *conjured directly*, as one assignment of the freed registers, along with every other state the design could conceivably hold. One step from there is examined, and the question becomes: does any state at all, under any input, step into a violation?

This move is what makes the proof both **complete** and **cheap**, and both for the same reason:

- **Complete**, because the arbitrary start *over-approximates* every reachable state. Whatever state a real run could ever be in after any number of cycles, that state is one of the freed assignments — so if no freed state can step into a violation, no reachable state can either, at any depth, forever. This over-approximation is the soundness argument in one sentence.
- **Cheap**, because the window is $K+1$ instants no matter how deep the interesting states live. The thousand-cycle walk is replaced by one constraint: “let the registers be anything.”

The price of the abstraction is that it is *too* generous: the freed registers can also take assignments no real run could ever produce — a FIFO whose pointers are three apart while its flags claim empty, say. Left unconstrained, such impossible states produce false alarms: violations that begin from a state the design can never be in. Taming that excess without giving up the abstraction is exactly what the rest of this chapter is about, and it is why the step *assumes the properties over the window*: the assumed set carves the arbitrary states down to the **compliant** ones, and — since every reachable state of a correct design is compliant — the over-approximation argument still closes.

So the two finite checks of **k-induction** are: the **base** — from reset, whatever the inputs do, the properties hold for the first K live cycles; and the **step** — from *any* state satisfying the properties for K cycles, the properties hold at the next cycle. Together: reset gives K good cycles, and the step extends any good stretch by one, hence forever.

19.3 The normal form

Version 1 wrapped this engine in ghost machinery: ghost-init files, closure checks per case, membership conditions. Version 2’s step is the textbook step and nothing else:

Tenet 3 — Induction is normal. The step is: *state free at the window start, the property set assumed over the window, the property set proven at the next instant*. No ghost inits, no closure cases, no membership machinery.

“State free” means every register and memory cell begins the window with an arbitrary value (data state as opaque tokens — Chapter 21). What confines this freedom is the **assumed property set**: the window’s cycles are constrained to satisfy every property. The step then asks whether a violation can occur at

the next instant. If clingo answers UNSAT — no such window exists — the properties are inductive, and with the base they hold for all time.

If clingo instead produces a window, look at it carefully, because it is usually not a bug — it is the abstraction’s known excess (Section 19.2) showing through. The window starts in a state that satisfies your properties but could never actually be reached, and from there one step misbehaves. The induction is telling you your property set is not yet strong enough to *exclude* that state from the compliant set. The response is to name the missing fact as a new property (for the FIFO family it is typically a ring invariant relating the pointers to the flags), add it to the set, and re-run. This doctrine has a name in the route:

A failed step is an invariant request, never a bug report.

Name the claim that excludes the impossible state; do not chase the counterexample as if it were reachable.

19.4 Two k’s, named apart

One vocabulary rule, paid for with a real alarm (journal step 18). The induction’s window width — the K of k-induction, `#const k` in the step’s generated program — is called **the window**. A stimulus file’s `#const k` — how many cycles a scripted story runs — is called **a horizon**. They are unrelated quantities that happened to share a letter, and reports and documents keep the words apart: at K=1 the step examines a 2-instant window, and no script and no walking are anywhere involved.

20. Scenarios: catching vacuity without scripts

20.1 The vacuity problem

An inductive certificate proves nothing *bad* happens. It is silent about whether anything *good* can happen. A design whose outputs are never valid satisfies “every valid output is correct” vacuously; a specification whose situations are contradictory satisfies everything. Version 1 fought this with scripted walks — drive this input sequence, then check the goal is reachable — and paid twice: scripts are expensive to reach deep states (“walk to full” costs exactly the depth-dependence induction exists to avoid), and a script checks one path where the claim is about a family of states.

Version 2 retired scripted stimulus entirely (the user’s call, journal steps 17–20: “*you remove it; if we need it, we can get it back from git*”). Its replacement borrows induction’s own trick.

20.2 Constrained abstract start, one cycle, an expectation

Tenet 9 — Anti-vacuity by scenarios. A scenario is a *constrained abstract start* plus one cycle plus an *expectation*: place the

machine in the interesting situation — compliant, meaning the properties are assumed, the same confinement the step uses — apply the input, and check the natural operation.

For the FIFO: “full, and a push arrives $\rightarrow$ the push is refused.” “Mid-fill, and both push and pop arrive $\rightarrow$ both happen.” Each scenario is one fact in the specification naming a state description, an input description, and an expectation. The runner performs **two solves per scenario**, both over the step’s own 2-instant window:

- **Solve A** requires the expectation to happen. It must be **SAT**: the situation is possible at all, *and* the natural operation actually occurs there. The witness is printed. If it is **UNSAT** the runner disambiguates with a third solve: either no compliant state matches the situation (your spec and design contradict — the situation is impossible) or the situation is fine but the natural operation never happens. The two diagnoses point at different files.
- **Solve B** forbids the expectation. It must be **UNSAT**: from *every* compliant state matching the situation, the natural operation cannot fail to happen. A SAT here is a real counterexample and is printed as a table.

Notice what the abstract start buys: “full” is *described*, not *reached*. The cost is independent of depth and history, and the claim quantifies over every compliant full state, not the one a script happened to construct.

20.3 Where independence now lives

A scoreboard-style reference model was also a second reader — a way for the specification to disagree with a wrong design. With the notebook gone, that independence is supplied by four mechanisms, each cheaper and each already in the flow: the **scenarios’ expectations** (a directed check the properties cannot satisfy vacuously); the **theorem structure** (first-in-first-out is a *consequence* of several independent local properties, so a wrong design must defeat them jointly); the **sabotage discipline** (every checker is demonstrated to catch a deliberately broken design before it is trusted — a checker without a sabotage witness is only believed to work); and, for the printed RTL, the **round trip** with a real simulator as the arbiter. That is Tenet 4:

Tenet 4 — No scripts. The v2 flow contains no scripted stimulus: the certificate is the induction plus the scenarios, both from constrained abstract states. Independence lives in expectations, theorem structure, sabotage, and the round trip.

An all-outputs-abstract level (v1’s “l0”) survives only as a diagnostic, not as a rung of the flow. The order of work on a level is fixed: **linkage and induction first — that is the certificate — then the scenarios.**

21. Data: never enumerated

21.1 Tokens and terms

The grounder’s cost is the product of candidate sets, so the methodology’s deepest rule is about what may have candidates at all:

Tenet 6 — Data is never enumerated, including by the proof machinery itself. Data positions carry opaque tokens and symbolic terms everywhere; any check whose grounding scales with a value domain is a defect of the check.

A 64-bit register freed at a window start is not 2^{64} choices; it is *one token* — `init(q)` — an opaque symbol about which nothing is assumed. Arithmetic over tokens builds *terms* (`add(init(q), 1, 64)`) rather than computing numbers. The certificate’s questions about control — does the conveyor move, is the full flag right — never needed the values; they need only that values route correctly, which token identity expresses exactly.

21.2 Derived arithmetic is enumeration in disguise

The rule covers the specification’s own vocabulary, in a way that is easy to violate innocently. ASP has no symbolic naturals: a specification symbol like `count(C, T)` with `C` ranging to a thousand *grounds a thousand candidates per instant*, and any rule joining two such symbols grounds their product. So:

- Prefer the design’s own **flags** and **wrap-bit pointers**: equality and successor over a domain of a handful of values. The FIFO’s specification does not track a count; it relates two pointers and reads `full/empty`.
- A counter that is not small **is datapath**: declare it data, let it be a token, and state the counting fact where counting is cheap — in Lean (version 1’s rule R5, carried).

21.3 The layered value story

What about the values themselves — the product of a multiplier, the datum a FIFO pops? The route answers in three declared layers, never blurred:

Tenet 7 — The value story is layered and stated. Routing is proven for all time where tractable; term identity is checked in composition, bounded; arithmetic is proven in Lean. What each layer does and does not establish is written down.

The **Lean half scales by changing shape, not effort**. At an enumerable width the theorem is decided outright — the 4-bit Booth multiplier’s correctness is a 256-case computation, with the deliberately unguarded variant proven *false* as a permanent control. At a real width the theorem becomes structural — for the 32-bit Wallace multiplier: a 3:2 compressor preserves the sum of its inputs; therefore a layer of compressors preserves the sum of the row list; therefore any

number of layers does; and the partial products sum to $a \times b$. Every 32-bit pair is covered by induction over the structure, and nothing is enumerated anywhere.

21.4 When Lean is needed — and when it is not

The route is sometimes read as “ASP plus Lean, always both.” It is not, and the dividing line is mechanical enough to state as a rule:

Lean is owed exactly when the certificate could not decide a value claim — that is, when a delivered-value obligation ends with two terms that differ as symbols. If no obligation was ever owed, no Lean proof exists to write, because the checker has already decided everything the specification claims.

The two cases, side by side:

- **Enumerable datapath: the checker decides outright.** The Am2901’s operands are four bits. Nothing in its entry is declared **data**, nothing is tokenized, and the certificate quantifies over the *actual values*: every one of the $16 \times 16 \times 2$ operand/carry combinations, under every function and destination, from every one of the 2^{68} states. When the step comes back inductive, the adder and the status equations are proven correct the same way the pointers are — by decision. The certificate’s report shows the fact plainly: **no obligation line appears at all**. Writing a Lean proof here would re-prove what is already exhaustively checked, which the budget discipline forbids.
- **Wide datapath: values ride as terms, and the debt is explicit.** The 32-bit multipliers have 2^{64} input pairs; their values cross the certificate as opaque parcels, and the delivery obligation compares the design’s closed-form expression against the specification’s @mul — two terms, differing as symbols. The verdict **OWED to Lean** is printed on the certificate’s last line, and the entry is not complete until the Lean theorem discharges it. The proof is structural, so it is paid once per *algorithm*: the compressor lemma, the recoding lemma and the disjoint-bits lemma served three multiplier machines.

Two refinements keep the rule honest. First, **independence does not vanish when Lean does**: on an enumerable entry the second-reader role is carried by the contract and the design computing the same functions in structurally different ways (the Am2901’s Figure-8 status stated at the word against the design’s gate-level cascade, their agreement itself proven inductive) and by the round trip’s simulator. Second, the rule is about *this entry’s claim*, not the device’s family: a **width-generic or compositional statement** — four slices cascaded into a 16-bit ALU, a parameter left symbolic — is a structural claim about all sizes at once, and that is Lean territory even when each concrete instance would be enumerable.

22. Composition: assume-guarantee

A system of modules is not proven flat. Each unit is proven inductive against its own contract **standalone** — where a small event-ghost is affordable if the unit’s contract wants one (the standalone exception of Tenet 2). The composed machine then *assumes* the units’ contracts instead of re-carrying their internals through the step; the composed bounded stories still check every contract unchanged, so an assumed contract is never an unexamined one.

Tenet 5 — Composition is assume-guarantee. Units proven standalone; the composed step assumes the contracts; the composed bounded legs re-check them.

The out-of-order pilot’s two execution units were the first test: both inductive at K=1 standalone, on the first attempt.

23. Names

Tenet 8 — Names are full words, by content, with hardware-style stages. No abbreviations, no mechanism prefixes. A signal or symbol carries a stage suffix — `<signal>_<stage>` — exactly when the stage is the distinguishing fact.

Version 1 prefixed ghost symbols with `g`. Under linked vocabulary the mechanism is visible from the linkage itself, so the name’s whole job is to say *what the thing is*: `grec` (a capture event’s residue) becomes `result` (what the machine holds as the outcome). The pilot’s full renaming, kept as the worked reference:

v1 symbol	meant	v2 name
<code>grec(S,V,T)</code>	the value the machine holds as slot S’s outcome	<code>result(S,V,T)</code>
<code>grd(S,R,T)</code>	slot S’s destination register	<code>destination(S,R,T)</code>
<code>ghead(H,T)</code>	the slot next to retire	<code>slot_retire(H,T)</code>
<code>gtail(L,T)</code>	the slot the next dispatch lands in	<code>slot_dispatch(L,T)</code>
<code>gcnt(C,T)</code>	how many instructions are in flight	<code>count(C,T)</code>
<code>ginfl(S,T)</code>	slot S holds a live instruction	<code>inflight(S,T)</code>
<code>gdone(S,T)</code>	slot S has completed	<code>complete(S,T)</code>
<code>gnd(S,T)</code>	slot S has not completed	<code>incomplete(S,T)</code>
<code>ggen(S,B,T)</code>	slot S’s generation bit	<code>generation(S,B,T)</code>

Most rows carry no stage suffix, because their meaning is stage-free; the two pointers get one because the stage is exactly what tells them apart. The rule binds everything new; v1 artifacts are renamed only when they migrate.

24. The certificate: what actually runs

`refine SPEC LEVEL --induct K` takes **no stimulus** and runs the whole certificate. What happens, in order (the code-level account, file by file, is `TOOL.md` beside this document):

1. **Lint and linkage lint.** The authoring subset is enforced; spec-side state outside a `refmodel` gate is refused; monitors over gated vocabulary are named as bounded-only. The gate is a **FACT** the bounded legs assert and the step does not: a rule carrying the literal `refmodel` is live in the base, the scenarios and the delivery obligation, and absent from the induction step (since 2026-09-05 – before that no leg asserted it, so a gated rule was inert everywhere and a gated `model` could never discharge; a field report’s probe found it).
2. **The base.** One solve from reset with every input free, over **K+1 instants** — reset occupies instant 0, so K *live* steps remain. This is not pedantry: the step’s “K exceeds the reachable diameter” argument leans on the base having actually examined K live steps, and a shorter base makes that argument unsound. (This path’s own sabotage witness caught exactly that bug during construction.) UNSAT means: from reset, no property can fire under any input sequence within the window.
3. **The step**, in normal form (Chapter 19): real registers and cells freed (data as tokens), `bad/viol` assumed across the window, a violation required at the next instant. Three solves guard the one answer: a **vacuity** solve first (the hypothesis alone must be satisfiable — otherwise “inductive” would be a green lie, and the runner says so instead); the hypothesis **with the unique-states constraint** (UNSAT here means K already exceeds the reachable diameter, which with the base is itself a proof); then the step proper.
4. **The scenarios** (Chapter 20), two solves each, with disambiguated verdicts.
5. **The delivery obligations.** Where the specification states a delivered value’s required form — `model(Port, Want, T)` with an `obligation_span(N)` giving the window the deepest lookback needs — one solve compares the design’s delivered symbol with the specification’s, and issues one of three verdicts: identical terms are **discharged by identity**; a symbolic difference is **owed to Lean** — recorded on the certificate’s last line, never miscalled a failure — and the Lean proof discharges it structurally; a concrete difference is a **violation**, with the witness table. An obligation that can never fire is a loud vacuity failure, not a silent pass.

Contracts are verified standalone by `contract <m>.lp --induct` (the composition prerequisite of Chapter 22). Every mechanism named in this chapter carries a witness test, plus a sabotage control wherever a check could pass vacuously.

24.1 The strong half, and the second configuration

Two additions the miss queue’s regeneration paid for (2026-08-31), now part of what a certificate IS:

The strong half. The standard induction step holds the reset released, and its report says so in one line. That line is a boundary: a monitor judged only where reset is ASSERTED — the reset-exempt family — can never fire in such a step, so “inductive” would be a vacuous claim about it. The runner now refuses to make that claim: with the reset pinned, reset-exempt monitors are reported **NOT EXERCISED in this step** and excluded from the inductive list; the certificate’s strong half re-runs the induction with `--free-reset`, where they genuinely bind. (Unmasking this on the miss queue exposed a real state the vacuity had been hiding — a stale in-flight bit in an invalid slot, read by an ungated count.) Scenarios stay under released reset in the strong half’s stead: a reset asserted mid-story cancels the story by definition, which the freed solver demonstrates and which is not a design defect.

The second configuration. A certificate at the built configuration is structurally blind to anything that only goes wrong under other parameters — a threshold baked at the default, a width a `DEPTH` override can shear. So the certificate runs at TWO points: the built configuration, and one off-default point (smaller, so cheaper) with the design regenerated there and the contract’s `#const` lines rewritten to match — and the gate asserts the point discriminates, by requiring the off-default contract to REJECT the default design. One extra cheap point turns “parameterized in name only” from an instantiation-time surprise into a certificate failure.

Live must be possible — asked before anything else. Every monitor the compiler writes is guarded by `live(T)`: with a reset, the instants where it is released; with no reset, every instant, because a block with no reset is a transition relation that is true from any state. The runner does not take that on trust. Before the base it asks the base’s own program one question — can ANY instant be live? — and refuses by name if none can. The reason is the shape of the failure it prevents: a contract in which nothing derives `live` has monitors that cannot fire, so the base is UNSAT, the step is UNSAT, and a false property is reported inductive for all time. That happened, on a reset-less block, and the only symptom was every scenario reporting “no compliant state”. The general rule the check instantiates is the one the route keeps relearning: before asking whether the bad thing can happen, ask whether anything can happen.

25. Gated datapaths: the `opaque_datapath.` directive

Production RTL gates its datapath: registers load only under a valid bit, operands are isolated to zero on idle cycles, so that a pipeline without traffic does not toggle. The first such design through the route (the production-form Booth multiplier, 2026-08-26) hit a wall worth understanding, because it will recur in any gated design.

A clock-enabled register can do one of two things each cycle — load or hold. An isolation mux likewise carries operand or zero. To the grounder these are *two candidate values per net per instant*, and it cannot see that all the forks are the same fork (they all follow one valid bit). A compressor tree then multiplies the independence: three rows joined per compressor make 2^3 combinations, whose outputs join again — 8^3 , then 512^3 — and the certificate dies writing the question down: 300 seconds, **Solving: 0.00s**. The bisect that convicted the gating (and acquitted the arithmetic changes landed the same day) took four one-line design variants and a 45-second cap each; the budget chapter’s recipe named the disease in one pass.

The cure is a declaration in the level:

`opaque_datapath.`

Under the directive, the **control solves** — base, step, scenarios — treat every internal data net as *one fresh token per instant*: the runner severs the data definitions and the data registers’ outputs from its copy of the design and lets the abstraction companion mint the tokens. This is sound because those solves’ claims are UNSAT claims (“no violation exists”) and tokens over-approximate every value the severed nets could compute: if nothing bad happens even with the datapath arbitrary, nothing bad happens with the real one. The **delivery obligation** — the one check that genuinely needs the computed values — instead pins the enable *inputs* (those wired to a data register’s enable pin) active as facts, so the grounder prunes the idle branches itself. That pinned path is exactly the path a delivery takes anyway: a delivery forces its own diagonal of valid bits.

A control input the datapath *reads* is a different animal, and it is handled the same way with or without the directive. Think of a multiplier’s subtract bit: it is not an enable, since both of its values deliver something, and the two deliveries are different functions. Pinning it active would prove the add and never look at the subtract. Leaving it free is worse in a way that is easy to miss: in the symbolic reading each net that reads the bit gets one candidate term per value of it, and the grounder does not know that thirty-two rows all read the *same* bit, so it grounds the cross product — a chain of adds over thirty-two rows becomes two-to-the-thirty-two ground candidates for one sum, and the solve never starts (this is how the 64-bit multiply-accumulate entry found it: at two rows, sixty-six

thousand rules; at four, no end). The runner therefore enumerates those inputs: it walks back from every data net’s definition and every data register’s data pin to the inputs, and runs the delivery obligation once per assignment of them, each held constant over the window. Each corner is exactly the single-candidate solve the pinned path always was, and the union of the corners is the whole control space, which is strictly more than one pinned corner ever covered. The report counts the corners and names the inputs; identity is claimed only when every corner is identity; a violation names its corner. Above two hundred and fifty-six corners it refuses by name rather than running for hours, because the count is exponential in the number of inputs the datapath reads — a design that far past the budget is asking a Lean question, not a runner question.

There is one shape of datapath the delivery obligation cannot evaluate at all, and it is the shape every production multiplier has: a *tree*. Clingo’s terms are trees with no sharing. An add chain’s term grows linearly, one row per link, and the 64-bit multiply-accumulate’s chain level certified in seven minutes. A compressor tree’s output at level eight carries a copy of everything below it — three to the eighth copies of every row — and every intermediate net is stored too; clingo aborted mid-witness on the first corner after writing 120 megabytes, with no error text, because an out-of-memory abort prints none. No width setting or budget reaches this: it is the representation. The route’s answer is the **obligation view**: the design declares `obligation_view(N, E)`, and the delivery leg reads the data net `N` through the expression `E` instead of its definition, dropping the definition and every definition only it reached. For the tree, `E` is the chain of adds over the same rows — the spelling the chain level proved against the specification — so the leg is linear again, and the one thing it no longer checks, that the tree computes what the chain computes, is reported and owed to Lean by name (`view(N)`), where it is the compressor lemma. The certificate’s claim is weakened by exactly one named theorem, never silently: the view is checked like a definition (a computed data net, declared leaves, the net’s width), the composer refuses it inside a child, and the print and the round trip never see it — the simulators still arbitrate the tree itself.

Boundaries of the mechanism, stated: data memories under the directive are refused by name (not yet supported); and a design whose *monitors deliberately read computed data values* — the FIFO’s cell checks — must not declare the directive, and does not need to: its storage sits behind one read mux, not a compressor tree, and grounds harmlessly. Do not reach for the directive on an ungated design, and never as a first response to slowness — measure first (Chapter 26); the directive is the cure for exactly this disease.

26. The performance budget: a rule, not a hope

A step that needs more than low minutes means something is being enumerated. Find it; do not wait it out.

Why enumeration is the whole story: clingo, the solver under the certificate, works in two phases. First it **grounds** — instantiates every rule over every value its variables could take, without knowing which combinations will matter — and only then does it **solve**, searching the result for answers. Grounding cost is therefore the *product* of the candidate sets a rule joins: one rule mentioning two free 8-bit values is 65,536 instances before any reasoning begins. The two times are reported separately, which is what makes the recipe below work — a run that spends its whole limit and reports **Solving: 0.00s** never reasoned at all; it drowned while still writing the question down.

The measurement recipe is two commands and has never failed to name the cost:

```
clingo -q1 --time-limit=120 <files>
# "Solving: 0.00s" at the limit => grounding, not search

gringo --text <files> | sed -E 's/[(:].*//' | sort | uniq -c | sort -rn | head
# who grounds: the head counts name the exploding predicate
```

One timeout is a diagnosis prompt. Two timeouts without a profile between them is a process violation (version 1's rule, kept verbatim).

One sample per port per instant is the compiler's duty, not the author's. A claim that reads several bits of one port — a neighbourhood, a window into a vector, a checksum over a word — reads the port several times. In every model those reads see one value, because a port has exactly one value per instant; the grounder does not know that, and if each read binds its own sample the join is the product of the port's domain with itself, once per read: 512^2 for two reads of a 9-bit word, and a neighbourhood of eight never finishes. The compiler therefore binds ONE sample variable per port per instant within a declaration and routes every read through it. The author writes the claim in the natural way. The cliff was found by reading the emitted rule, not by timing the source, and that is the method for any such cliff: the artifact says what is joined; the source does not.

27. Generated designs

Real datapath RTL comes from generators, and so may a level: the Wallace multiplier's 11.1p is emitted by a small committed `generate.py` (word-level carry-save arithmetic keeps 32 bits at roughly a hundred definitions). Three rules make this safe:

1. The generator is committed beside its output, and the output is **regenerated, never edited**.

2. The lint gates the emitted file exactly like an authored one — it caught the generator’s bare shift-amount constants on the first run.
3. The Lean model mirrors the generator’s **algorithm** (the same triple-grouping rule), so the proof is about the same reduction schedule — a by-inspection link, stated where it lives.



Part IV — Practice

27.1 What the printed RTL must look like

The route’s output is RTL an engineer will read, so the print has conventions, each of them a ruling from the miss queue’s regeneration run (2026-08-31) and each now carried by the printer rather than by discipline:

1. **Parametric, honestly.** Sizes are `#(parameter ...)`, entry state is packed arrays over the parameter, and — the part that is easy to miss — every `COMPARISON` against a size is a parameter expression too: `liveCount < COUNTWIDTH'(DEPTH)`, never a baked number. A module parameterized in its header but comparing against the defaults is parameterized in name only.
2. **Grouped generate for blocks.** One block carries a cohesive cluster — a slot’s CAM nets together, all of a slot’s flops together — not one block per net, which is the same machine-look one level up.
3. **The staging convention.** Every register `xx` is fed by a combinational `xxM1` carrying its complete next value with the hold term visible (`set | (hold & ~clear)` for bits, a single-level mux for payloads), and the flop is `xx <= xxM1` — no write-enable pins hiding the hold inside a primitive, no vestigial `if (1)` guards.
4. **Names are for people, here too.** lowerCamelCase, no numbered temps, no hoisted `__e` wires (the printer inlines width-safe subexpressions and casts constants at parametric widths), and no `@` sigil in any identifier — the sigil stops at the surface. The `@` in `always_ff @(posedge clk)` is SystemVerilog’s own event control.

What “width-safe” means, exactly (2026-09-04, from a review of the Life print, where one eight-term sum had become six wires). SystemVerilog sizes an expression to its context — the target of the assignment, or the widest operand around it — and never truncates in the middle. The `@funcs` do truncate, at the width written on every node. The two agree whenever the node’s width *is* the context’s width and the operation is one for which reducing modulo 2^w after each step equals reducing once at the end: add, subtract, multiply, and the bitwise and/or/xor. Such a node is

printed inline, in parentheses; a select `x[i]` is always a leaf; and `b ? 1 : 0` at width `w` prints as the cast `W'(b)`. A node whose width differs from its context — `add(add(a, b, 4), c, 8)`, “add at four bits, then widen” — keeps its wire, because there the intermediate truncation is the semantics and an inline `a + b` would keep the carry the four-bit sum drops. The gate `print_still_hoists_a_sum_narrower_than_its_target` is that case, round trip included. The flop generate borrows the loop-variable names of the lane feeding its `d` pin, so `g[r][c] <= gM1[r][c]` reads beside the generates that use `r` and `c`.

The round trip is what holds the printer to all of this while keeping it honest: the printed file must translate back and match the authored model value for value, with Icarus arbitrating — a print convention that broke the round trip would be rejected by the harness, not shipped.

A lane of bits is one word, and it prints as its own name. A `net_lane(cap, N, 1)` is declared logic `[N-1:0] cap` in the print, so the packed word *is* the lane’s name. In the design that word is written `pack(cap)` — explicit rather than a bare lane name accepted where a net is expected, because a lane of wide elements has a packed word too and its cost is exponential in the element count; declared, it can be budgeted. The printer emits `assign out_byte = cap;`, one line, parametric by construction. The alternative — assembling the word arithmetically, one weighted term per lane — is a fake parameterisation, which is how this convention was found: the route’s print-parity check failed on exactly that chain.

27.3 A regular relation is a lane, and a grid has two axes

A grid, a ring, a butterfly, a systolic array — a block whose wiring is a COM-PUTED relation between positions — is written as lanes whose index expressions carry the relation, never unrolled by a generator into hundreds of literal cells. The generator produces a module that says `$(parameter SIDE = 16)` and then carries 256 hand-wired cells with baked indices; at any other side it is wrong, and the print-parity check will say so in thousands of lines.

A lane has as many axes as the structure has. `net_lane(g, (side, side), 1)` declares a grid whose members are `g(r, c)`, in row-major order; `def_lane(y, (R, C), e)` binds one loop variable per axis; a reference indexes each axis on its own, and each axis wraps on its own, `\ B` being the ring wrap over the axis’s extent. Conway’s eight neighbours on a torus are therefore eight one-line definitions with no row base, no linearisation and no special case for an edge or a corner:

```
def_lane(up, (R, C), q(R - 1 \ side, C)).           % the row above, wrapping
def_lane(dn, (R, C), q(R + 1 \ side, C)).
def_lane(lf, (R, C), q(R, C - 1 \ side)).           % the column to the left, wrapping
```

```

def_lane(rt, (R, C), q(R, C + 1 \ side)).
def_lane(ul, (R, C), q(R - 1 \ side, C - 1 \ side)).    % the diagonals, one wrap per axis
def_lane(ur, (R, C), q(R - 1 \ side, C + 1 \ side)).
def_lane(dl, (R, C), q(R + 1 \ side, C - 1 \ side)).
def_lane(dr, (R, C), q(R + 1 \ side, C + 1 \ side)).

```

A grid reads a FLAT wide port per cell through a position over its own loop variables: `def_lane(seed, (R, C), bit(data, add(mul(R, side), C)))` prints `data[((r*SIDE)+c)]`, the translator reads it back as a lane read at a computed position, and the port stays flat, so a contract compiled from the flat signature keeps reading `data[C]` (G30).

The state of a grid is a lane of registers: `inst_lane(uG, ff, (side, side))` with `pin(uG, d, gM1)` and `pin(uG, q, g)` joins each cell's pins to the members at its own position, and prints as one nested generate with the cell once, `g[i][j] <= gM1[i][j]`. That is the one primitive a grid cannot do without, and with it Conway's Life is a specification's worth of lanes: the eight neighbours, the count, the rule, the grid held in the flops, the load path.

The offset and the wrap block may each be a NUMBER or a PARAMETER. The loader resolves them to unroll for the solver; the printer keeps the names, so the module declares `logic [SIDE-1:0][SIDE-1:0] q`, nests one **generate for** per axis, and reads `q[((r + (SIDE - 1)) % SIDE)][c]` — the same text at every side, which is what print parity asks; a literal offset against a parameter block prints as `(SIDE - 1)`, never as the residue it happens to be at one size. `pack(g)` is the grid as one word, row-major, the order in which SystemVerilog flattens packed dimensions and in which the translator reads a nested generate back.

Why this is not a convenience: linearising a grid onto one index (`q(I - side \ cells)`), the column wrap written as a wrap *within a block of size side* puts row-major order into every neighbour expression, where every reader — the author, the lint, the printer, the translator reading the print back, the harness comparing members — has to reverse it. Worse, it hides the two facts the argument needs. To the grounder, an index written as arithmetic is a computation whose result ranges over values, instantiated and joined; a position written as `(R, C)` is a term, and a rule over `g(R, C)` is one rule at every cell. To the induction, a cell's next value depends on its eight neighbours and a property about a cell mentions the same eight; written per axis, that locality is in the terms and the step closes at $K = 1$ as the stencil did, while linearised it is buried in arithmetic the solver cannot see through. Chapter 34.6 draws the consequence for the certificate.

What the vocabulary still does NOT offer is a general index expression — a stride `2*I`, an exclusive-or with a constant (the butterfly pair), an index read from another lane — and that stays recorded rather than built, because every form added here must also print as a **generate for** the translator reads back, and only a block that needs it can say which form.

27.5 The simulator, and how a 2-state one is kept honest The round trip’s last leg simulates the printed RTL and compares every sample with the authored model. Two simulators can do it, and the tool takes whichever the machine has, preferring Verilator (`--sim auto; --verilator` and `--icarus` name one outright). They differ in one way that matters to the comparison. Icarus is *4-state*: a value that depends on state nothing has set yet prints as `x`, and the runner simply does not count that sample. Verilator is *2-state*: the same value prints as 0, indistinguishable from a real zero. A plain swap of the compile command would therefore compare power-on garbage against the model’s power-on convention and report agreement that means nothing.

The rule that keeps Verilator honest is the **two-fill rule**. The bench is compiled once and run twice, the first time with every unset bit at 0 and the second with every unset bit at 1, and a sample counts as *definite* only where the two runs agree. A value that depends on power-on changes with the fill and is skipped, which is exactly the decision Icarus takes with `x`. On the FIFO both arbiters skip the same sixteen samples (the unwritten memory cells) and compare the same 182, and the gate `fifo_round_trips_under_verilator` asserts that the two counts are identical.

The rule is an approximation in one direction only. A value such as $q \hat{=} q$ on an unreset `q` is `x` in 4-state and 0 under both fills, so Verilator will compare it where Icarus would not; no sample Icarus would compare is ever skipped. The report names the rule in use (`x/z in the 4-state simulator`, or `the all-zeros and all-ones power-on fills disagree`), and, as before, a comparison over no definite sample at all is a failure, never agreement.

28. What an entry is

How it is built — one requirement at a time. The construction protocol is incremental by rule, never one-shot: for each requirement, the author (or the tool acting for them) presents (a) the requirement as a causal sentence, (b) the window or windows it needs — with their prose meanings — and (c) the condition that will check it; the user reads and approves; only then is it added, and only then does the next requirement begin. A complete contract may be generated in a single pass only when the user specifically asks for one. The reason is the seam warning of chapter 3: races live between causal sentences, and the moment a new sentence is laid beside the existing ones is exactly when its seams are cheapest to see. One-shot generation produces a contract nobody watched being assembled — which is how the first miss-queue contract shipped a contradiction that only the certificate could expose.

An entry, when it closes, consists of:

1. **SPECIFICATION.md** — the English in force: the adversarial-misreading pass done, every ambiguity resolved in a numbered record (R1...), every checkable sentence tied to the promise that checks it, structural mentions justified, freedoms listed.

2. **The contract** — causal sentences where forms fit, their committed expansions inline, window declarations, promises in violation form (one named **bad** per way of being wrong), scenarios doubling as goals, the environment’s assumptions marked as such.
3. **Per design: the linkage** — the adapter mounting the windows on that design’s registers (this is where a designer meets registers; the spec author never does).
4. **Certificates** — the time induction at the base configuration; the size-window certificate when the structure is parameterized; units standalone before composition.
5. **The concrete leg** — the print, and the round trip against the translator with a simulator arbitrating.
6. **Sabotage witnesses** in the suite, and — where the entry claims generality — the freedom witness.
7. **DESIGN.md** — the decisions with their reasoning, the alternatives rejected, what the entry cost the tools, and what remains open, in the house register.

29. The no-regress chapter

Every line below was paid for by a specific failure. Most of them are the chapters of Part III in one sentence each, gathered here so that the whole set can be checked against a proposal at once; a few — double induction, the three layers, the boundary-spelling hazard, the process rules — appear only here. **A proposal that conflicts with a line below needs to argue against the incident that created it, not just against the rule.**

- **Windows, not ghosts.** The contract’s state vocabulary is *declared windows*, mounted by each design’s linkage on that design’s real registers. No invented spec-side state outside `refmodel`. Why: a ghost is a copy, a copy needs glue invariants, and the glue is what inductions choke on — v2’s fastest proofs (the Am2901, the miss queue: **zero hand-written invariants**) are fast *because there is no copy*.
- **Normal-form induction, and what a failure means.** The claim engine is k-induction in normal form: state free at the window start, the property set assumed over the window, a violation sought at K. Start at K=1; raise K before writing an invariant. A failed step is an **invariant request** — it names a missing confining claim — not a bug report, and not a request for a ghost.
- **Double induction, and never enumerate an instance family.** Time induction proves “for all time”; **size induction** (a window certificate: a small generic instance whose surroundings are free, promise-constrained summaries at the design’s fold interface) proves “for all sizes” in one solve. The user’s rule, verbatim: *“I don’t want you to run manually for those sizes — then the purpose of the induction will be lost.”* Per-size certificate tables are never evidence.

- **The three layers.** Pure-function units (a CAM answers “are these equal?” and nothing else) / POLICY (what an answer means; **every race decision is a policy conjunct**) / pure STATE (every event arrives decided). The layering is not mandated by contracts — it is how designs stay provable and honest.
- **Datapath facts go to Lean once.** An arithmetic identity inside a unit (the XOR-NOR tree computes equality) is proven in Lean for every parameter and assumed by the ASP library as a consistency fact — the gate-spelling constraint with `RouteLean.Cam.comparator_is_equality` as its license is the model. The two-prover split otherwise stands: control to clingo, arithmetic residue to Lean, and an entry whose obligations all discharge by identity carries **no Lean at all, by rule**.
- **Assume-guarantee is the composition.** Units proven standalone against their own contracts; the composed certificate *assumes* abstract children’s guarantees and **proves** their requires; contract data outputs are stated in `model` form (an abstract data net is a fresh token per instant unless a model speaks).
- **Reset-once.** Reset before time 1, never again — the standard convention, enforced by the machinery (the base drives reset only at $T=0$; the step holds it released).
- **Scenarios and sabotage are the teeth.** Every specification ships reachability scenarios whose natural operation must be unviolable — they have caught every silently-vacuous proof so far (including an entire composed certificate that was green only because its abstract instances were dark). Every claimed fix or detector gets a **sabotage witness** that discriminates: break the thing, watch the exact check fail, restore.
- **The vacuity discipline generally.** A conformance check must require its goals to be reachable, or “no violation” and “nothing is possible” are the same answer. An empty check says so; it never passes.
- **The boundary-spelling hazard.** Two spellings of one question (“gate equality” vs “ideal equality”; `eq(B, k(0,32))` vs `eq(B, 0)`) become two independent free bits unless a stated fact ties them. Any new spelling pair needs its tie, and the tie needs its license (a Lean lemma or a library-level identity).
- **Measure before fixing; one timeout means profile now.** Solving: 0.00s means grounding; bisect the artifact before theorizing. A remembered fix is a hypothesis. And never mix a tool change with entry work — tool changes land as their own gated commits first.
- **Minimal verification.** Scope every check to what the change could affect; a full certificate is run once, when the artifact is believed done — never as the iteration loop.
- **Naming and register.** Full words, content names, stage suffixes only where the stage is the distinguishing fact. Every read-back and document at the human register this document is written in.

30. What carries over from version 1 unchanged

The authoring language and rule library (`lib/aspfirst/` — shared, not forked). The printer and the round trip, including `--incremental` (the per-instant chained solve for translated sides whose direct grounding cross-products candidate values). Resolutions-before-ASP and the controlled specification grammar. The contract stated over observables. Goals-must-stay-reachable as the anti-vacuity floor. The human-gated ladder (pending $\rightarrow$ built $\rightarrow$ explained $\rightarrow$ approved $\rightarrow$ verified; only the user sets approved) — carried over in name only until 2026-08-31, when it was actually implemented; see Chapter 32. The claim taxonomy (proven / checked-by-construction / asserted-by-a-human). And hard rule 9: archive, never delete.

31. Status, migration, and holds

- **The v1-shaped release is retired outright** (the user, 2026-08-25: “the previous release version is bad, we don’t need it anymore”). Its kit is archived at `archive/spec2rtl_v1/release/`; a future release will be designed fresh, v2-shaped, when the v2 corpus warrants one.
- **State (2026-08-27)**: eight artifacts complete under v2 — the FIFO, `ve129_seq101`, `rtllm_traffic_light`, `booth`, and the multiplier family `wallace32` / `booth_wallace32` / `booth_production32` (one contract, rule-identical three ways and gate-enforced; three machines, each certificate-inductive, round-tripped under Icarus, and product-theorem proven in Lean). The work queue: the remaining dataset entries and examples from their retained specifications, then the out-of-order stages A and B. `LEARNINGS.md` beside this file carries the paid-for lessons; `JOURNAL.md` records every step and every intervention; `TOOL.md` explains the implementation file by file.
- **Migration**: v1 entries migrate one at a time, each fully re-certified under v2; v1 code and remaining artifacts move to `archive/spec2rtl_v1/` when nothing green depends on them. v1’s operating documents are already archived; the maintainer history (`notes/design/ASP_FIRST_LESSONS.md` and companions) and the two books stay in place as records of what was done and found.

31.1 What is machinery today, and what is still owed

Carried and working: the v2 tool (`aspfirst2` — normal-form induction, contracts, abstract composition with clock/reset hints, the model form, the export, the roundtrip), the authoring library with the equality theory (generative symmetry, single-spelling transitivity, the gate link), and the CAM comparator identity in `lib/lean`.

Owed, in order (tracked in `notes/WORKLIST.md`):

1. **The window-certificate machinery** — size induction as a *gated capability* of the route (generic window + summary interface + the drift check pinning the interface to the generator’s structure), not a per-entry improvisation.
2. **The forms library** — the initial causal forms with their committed expansions and, per form, its documented boundary instants.
3. **A spec lint** — the avoid-list (implementation-flavored vocabulary in a contract) as soft warnings.
4. Someday, an **SVA export backend** — the industrial bridge, generated from the contract, never the other way around.

32. The ladder: every artifact is a step, and a person approves each one

The route’s artifacts are built in order, each derived from the one before it: the English, the signature, the specification in the language, the contract, the design, the certificate, the RTL. An agent can produce all seven without stopping — and what it hands back is then a finished thing nobody watched being assembled.

That is not a hypothetical failure. The first miss-queue contract shipped a contradiction that only the certificate exposed, because its clauses were laid down together instead of one at a time. Races live in the seam between two sentences, and the moment a new sentence is laid beside the existing ones is exactly when its seam is cheapest to see.

Tenet 9 — The route is a ladder, and a person stands on every rung. Each artifact is built, explained in plain language, and *approved by the user* before the next one begins. An agent never sets approved.

The ladder lives in the entry, as `ladder.yaml`, with five states per step:

```

pending      the step has not been done
built        the artifact exists
explained    its read-back has been written -- what it says, in plain language
approved     A PERSON HAS READ IT. Only the user sets this.
verified     the step's mechanical check has passed (where the step has one)
```

`python -m sv2asp.aspfirst2 ladder status <entry>` prints the ladder and names the next action; `ladder built` and `ladder explained` move a step. **There is no ladder approve, and its absence is the design** — the tool has no code path that writes `approved`, so an agent must forge it deliberately rather than reach it by running a command.

What this guarantees, and what it does not. It is auditability, not enforcement: nothing here can stop a misbehaving agent from writing `approved` into the file, and claiming otherwise would be exactly the kind of false claim

this route exists to prevent. What it does give is three things. The state is a committed file, so “*did a human actually see this?*” is answerable months later. The tool refuses to advance a step whose predecessor is unapproved, so the ordinary path is the disciplined one. And — the part with real teeth — **an approval records the digest of what was approved**, so editing the artifact afterwards makes the approval *stale*, reverts the step, and shuts the gate again. That last rule catches the innocent version of the failure, which is also the common one: approved, then quietly improved.

Witnesses: `test_v2_ladder_gates_each_step_on_the_previous_approval`, `test_v2_ladder_approval_goes_stale_when_the_artifact_changes` (sabotage-verified: with the staleness rule disabled, the test fails), and `test_v2_ladder_refuses_a_malformed_file` — a typo silently ignored is a gate that silently did not run.

Do not confuse it with `verify.json`. That is the *verification* flow — which checks run for an entry, with what parameters (Chapter 24). The ladder is about who has read what. Two unrelated things called “flow” is how a reader ends up believing a check ran, which is why version 1’s `flow.yaml` was renamed here.

33. The four semantic decisions

A language can be perfectly usable while several of its rules live only in the author’s head. You write a specification, read it back, and it says what you meant; the conventions hold because one person applied them consistently.

A compiler ends that arrangement. It cannot ask what you meant. It picks one reading and makes it law for every specification written afterwards — and if that reading differs from the one in your head, nothing crashes. The specification compiles, the certificate passes, and the difference surfaces as **a clause that quietly checks nothing**.

Three of the four decisions below have exactly that failure mode. They were settled on 2026-08-31, after a review of the miss-queue entry found each of them live in a real file. Each is recorded here with its ruling, the alternatives that were rejected, and — the part that makes a decision real rather than documentary — **the check that catches it being broken**.

33.1 What an object is: the parking space, not the car

A car park has four spaces and a rule: *a car here more than two hours gets a ticket*. Space 2 holds a car for ninety minutes; it leaves; another parks. Thirty minutes later, has “the car in space 2” been there two hours? Tracking **spaces**, yes — and you ticket an innocent driver. Tracking **cars**, no. Same rule, same facts, opposite answers.

The miss queue has four slots, and an entry is created, lives, is answered, and ends — after which that slot holds a different line. So **each entry E** must mean one or the other.

Decision: an object is a SLOT. Identity is the place; **exists(E)** says whether it is occupied, and every quantifier ranges over the objects that exist at the instant it is evaluated.

Why: it is what the hardware is. This route stays cheap by reading the design's own state rather than building a parallel picture beside it, and episode identity is a parallel picture by definition — it would make the design carry a generation number for no reason of its own.

Rejected: episode semantics, where each **create** mints a fresh identity. It removes the car-park ambiguity outright, and it was rejected only because of the cost above. If a future design genuinely needs to talk about an entry after it has ended, this is the decision to revisit.

The hole it leaves, and the rule that closes it. Slot semantics makes a time-spanning claim ambiguous: a rule saying *entry I is answered within four cycles* can be satisfied by a **different** entry that moved into slot I meanwhile. One request's obligation discharged by its successor's service. The miss queue's own contract contained such a rule, sound only by an accident of how entries end.

The check: a rule that mentions a bound object at two different instants must also say whether it still exists at the later one. A property referring to **E** under a delay without mentioning **exists(E)** there is refused by name.

And one consequence worth stating out loud: because quantifiers range over what exists, **every quantifier silently asks the design which slots are live.** **exists** is therefore a window like any other, and it joins the mount manifest whether or not the author writes it.

33.2 Quantifier scope: say it, do not imply it

“If someone left a light on, turn it off.” Ordinary English — and the first half gives you only the fact **that** a light is on. It never hands you the light. The second half points at something the first half did not keep, and a human listener silently repairs it.

The honest versions are *“turn off **every** light someone left on”* (stronger) and *“if a light is on, something is wrong”* (weaker). The original sits between them and commits to neither.

The miss queue had exactly this:

```
some entry E where inFlight(E)
  |-> s_eventually(fill.arrives(F) && F.address == E.address)
```

meant as *every* in-flight fetch is eventually filled — a real promise about memory — but written in a form that also reads as the near-worthless one. Both spellings look correct.

Decision: every quantifier carries an EXPLICIT SCOPE, and a variable exists only inside it. Inline, the scope is parenthesised. Over several claims, the scope is a block, and every claim in the block speaks about **the same instance**.

```
@assume memoryEventuallyAnswers
  each entry E ( inFlight(E) |-> s_eventually(fill.arrives(F) && F.address == E.address) )

each entry E:
  @property addressIsStable
    exists(E) | => !exists(E) || $stable(E.address)
  @property demandForwardedAfterFill
    exists(E) && filled(E) && demanding(E)
    |-> ##[1:depth] ( (exists(E) && forwarded(E)) || !demanding(E) )
```

Why the block form matters as much as the parentheses: the interesting claims about a miss are not one sentence but several, and they are only meaningful **about the same miss**. Before this, each property re-bound its own variable and the shared subject was a convention. Now it is visible in the shape of the file.

Rejected: forbidding a variable to cross the implication at all, which was the earlier recommendation here. Explicit scope is strictly better: it permits the reading authors actually want — universal quantification over a whole implication — while making “same instance?” answerable by looking rather than by knowing a rule. *Also rejected: witness semantics*, where the antecedent’s **some** retains whichever object made it true, because when several objects qualify there is no natural answer to which one is meant, or whether it may change between instants.

The check: a variable used outside its quantifier’s scope is refused by name, with the enclosing scope suggested.

And some turns out to be doing two different jobs, which is the single most useful thing the second review found. Compare:

```
!some entry E where E.address == R.address && joinable(E)      -- a PROPOSITION
some entry E where joinable(E) && !demanding(E) ( ... )        -- a WITNESS BINDER
```

The first asks a question about the population and answers true or false. The second says “*take an entry with these properties, and here is what the rule says about it*” — and if no such entry exists, the rule simply has nothing to say.

Written without noticing the difference, the second reads as the first, and the consequences are severe. Two behaviours in the miss queue said, in effect, “*every demand requires a matching prefetch to exist*” and “*whenever the fetch limit*

allows, some entry must be eligible” — the first unsatisfiable on an empty queue plus a new demand, the second unsatisfiable at reset. Neither was what the author meant, and both read perfectly well.

The rule: a some with a scope is a WITNESS BINDER, and the scope is what it binds over. A some without one is a PROPOSITION. The scope is therefore not decoration — it is what distinguishes the two meanings, and it must enclose the whole rule the witness participates in, trigger and effects alike.

The practical form: when a rule is *about* a particular object, the quantifier goes **outside** the trigger, not inside the effects:

```
some entry E where joinable(E) && !demanding(E) (
  request.valid(R) && R.isDemand == 1 && E.address == R.address
  -> accept R
  ##1 demanding(E) && E.tag = R.tag
)
```

“For an entry that is joinable and not yet demanding: if a demand arrives for its line, take the request and lift the entry.” When there is no such entry the rule is silent, and some other rule — here, allocation — handles that case.

One carve-out, stated so it is not mistaken for an exception. An interface predicate that binds a payload — `request.valid(R)`, `forward.valid(D)`, `fill.arrives(F)` — is **not** a quantifier and needs no scope. It names *the thing on that interface at this instant*, of which there is exactly one, so it is closer to giving something a name than to searching for it. Such a binder is in scope for the whole rule, antecedent and consequent alike. The distinction that matters: a quantifier **searches** a population and may find several, which is why its scope must be visible; an interface binder **names** a single thing that is simply there.

some remains available and remains common — *“no entry holds this line”* is a plain yes-or-no fact and needs no scope beyond itself. The rule bites only where a claim reaches back for the object that made a condition true.

33.3 Correspondence: the courier labels the parcel

A courier hands you a parcel. Which of your three outstanding orders is it? You cannot always tell from the contents — two orders may hold the same item. The reliable answer is the label, and **the sender attaches it**. The recipient does not get to decide afterwards what the parcel was for.

When the miss queue answers a request it drives a valid, a tag and a line. Four claims then speak of *the entry this answer belongs to*: that it was genuinely waiting, that the tag matches, that the data is the right line, that it was not a prefetch. Nothing said which entry that was. The relation lived in a comment while four claims leaned on it. And it cannot be recovered by matching tags:

tags come from outside the block, nothing promises they are unique, so “*the entry whose tag matches*” may name the wrong one.

Decision: the DESIGN names the correspondence, and the specification reads it as a window. A `send` behaviour declares the object it serves; the compiler adds the corresponding window to the mount manifest, and the level’s linkage mounts it on the design’s own selection signal.

`send on forward as D answering E with tag = E.tag, data = lineData(E)`

One clause, three bindings. `as D` names the event so claims can speak about it, and the clause establishes all three of `forward.valid(D)`, `corresponds(D, E)` and `forwarded(E)` at that instant. Before this the specification used all three and created none of them: `D` was a name with no binder, and `forwarded` and `corresponds` lived in prose while four claims leaned on them. The `answering` half is the demand on the design; the `as` half is what makes the result speakable.

Why: it costs the hardware nothing — something already had to select which entry to answer — and it turns the relation into something **checked** rather than asserted. “*An answer only ever goes to an entry that was filled and demanding*” becomes a real claim with teeth.

Rejected: **letting the specification establish the relation** (establish `corresponds(D, E)`). Two objections, the second decisive. It is specification-side state, a small instance of the bookkeeping this route exists to avoid. And a declaration is not a check: the specification would *announce* which entry an answer belongs to with nothing confirming the design agrees, at exactly the point where four claims depend on it.

The scope of the ruling. This works because the interface answers **at most one request per instant**, which makes “the answer happening now” well defined; `iface.valid(D)` already binds that payload. An interface issuing two answers in one cycle would need genuine event identities, and none of this carries over.

The check: a `send` behaviour whose object is referred to by any claim must name it. The named window joins the manifest, and an unmounted one is reported like any other.

33.4 Reset scope: the fire-drill rule that cancelled the fire-drill rule

An office has a blanket policy: *nobody is assessed during a fire drill*. Someone then writes: *during a fire drill, everyone must be outside*. The second rule can only be checked during a drill; the first says nothing is checked during a drill. The blanket exemption silently cancels the one rule written for that situation — and nobody notices, because a rule never checked and a rule never broken leave identical records.

The miss queue’s specification opens with `disable iff (!resetN)`, and then states two properties that fire only while reset is asserted. For an entire working

session those two clauses looked like guarantees and checked nothing. They survived only because the hand-written contract expressed them differently and *was* checking them — so two files meant to say the same thing disagreed about whether two of their clauses existed at all.

Decision: a property that mentions the reset signal is EX-EMPT from the file-level disable iff. No ceremony, no override clause.

Why: it makes the common case correct by default, and the case it covers is unambiguous — a property naming the reset signal is, in practice, a property about reset.

And the implementation must be that sentence, not a proxy for it. The compiler first granted the exemption to one SPELLING — the `@during reset` phrase, which reaches it by a different internal route — rather than to a property naming the reset. On the miss queue the two coincide, so nothing showed. On a block whose author wrote the same requirement as `when reset is high, then ...`, the claim compiled into a monitor requiring the reset to be both asserted and released at one instant: present in the contract, counted among the monitors, and unable to fire under any execution. Behaviours had no exempt form at all, which is what left “after a reset cycle the phase is idle” — reset-exempt *and* about the next cycle — with no sound spelling in the language. The exemption now follows the meaning and covers behaviours as well as properties.

Rejected: **an explicit per-property override** (`enable iff`), which is the conventional answer and was the earlier recommendation here. It is more explicit but adds ceremony to every reset property, and ceremony that is usually omitted becomes a defect generator.

The objection to the ruling, and its answer. An implicit exemption means a reader cannot tell, by looking at one property, whether it is live or dead — the rule has to be simulated mentally. That objection is real and is answered not by the scoping design but by the check below, which makes every silenced property visible in the report. **Under any of the three designs a property can be silenced without a word; only the report fixes that.**

The check: every certificate reports the properties that **never had a chance to fire** — disabled at every instant of the run. A silenced property announces itself instead of hiding, exactly as a window with no glass does.

33.5 What the four have in common

Three of them — object identity, quantifier scope, reset scope — fail the same way when guessed wrong. Nothing errors, nothing turns red, a sentence quietly stops meaning anything, and the certificate reports success.

That is why each decision above ships with a check rather than only a definition.

The definitions alone would be documentation; **the checks are what make them the language.**

34. Proving for every size: the depth question

A queue is written with a depth. The certificate is run at one — four, say — and comes back green. The specification, meanwhile, is written parametrically: it says `##[1:depth]`, not `##[1:4]`. So the certificate proves something narrower than the specification claims, and the gap is invisible because both artifacts use the same word.

This chapter is about closing it. It is placed after the semantic decisions because it has the same character: what goes wrong is not a failure but a claim that quietly means less than it says.

34.1 The shortcut that is not allowed

The obvious move is to run the certificate at four, at eight, at sixteen, see green each time, and conclude. **This route forbids it, by rule**, and the rule is worth restating in the user’s own words: *“I don’t want you to run manually for those sizes — then the purpose of the induction will be lost.”*

The reason is not fastidiousness. Time induction earns “for all time” by proving a step that holds from *any* state, which is why it does not walk. A table of sizes walks. It also happens to be weakest exactly where designs break: the miss queue’s forwarding arbiter is correct at depth four and starves at depth five, and a table stopping at four would have reported success about a design with a starvation bug in it. That is not hypothetical — it is what this entry did until a review caught it.

34.2 The properties divide in three, and only one third is hard

Reading the miss queue’s thirty-two promises with the question *“what does this depend on?”* sorts them cleanly:

Local — one slot and its inputs. `slot_phase`, `address_disturbed`, `want_disturbed`, `inflight_disturbed`, `data_disturbed`, `demanding_disturbed`, `entry_appeared`, `entry_vanished`, `fill_lands`, `enquiry_clears_want`, `redirect_downgrades`, `prefetch_ends`, `want_never_returns`, `forward_is_correct`, `forwarded_entry_is_freed`, and the reset pair. Nothing in these mentions another slot. They are the majority.

Pairwise. `one_entry_per_line` relates two slots and nothing more.

Fold-dependent. `never_overfilled`, `fetch_limit_held`, `demand_always_has_room`, `ready_is_right`, `occupancy_wrong`, `inflight_count_wrong` all rest on a **count** over the slots; `demand_first` and `fetch_is_real` rest on a **priority**

`select`; `forward_is_real` and `demand_forwarded_after_fill` rest on a **rotating select**. These are the ones that genuinely depend on how many slots there are.

That division is the whole plan. Two thirds of the property set never mentions depth, so it does not need an argument about depth — it needs a way to say “one slot, arbitrary surroundings” and prove it once.

34.3 The window certificate: one slot, promises for the rest

The mechanism for the first two groups is a **window certificate**: instantiate a single generic slot, leave the rest of the queue as free values, and constrain those values by the promises the folds make rather than by how they are computed. The three fold interfaces and what each promises:

fold	what the design computes	what the window assumes
count	popcount of the valid (or in-flight) bits	a number in $0..depth$; zero exactly when no bit is set
priority select	the first set bit	at most one selected; a selected slot has its bit set; if any bit is set, something is selected
rotating select	the first set bit from the pointer	the same three, plus: the pointer advances by one after each grant

A property proven against those assumptions holds at every depth, because nothing in the proof ever counted the slots. What the window certificate does **not** discharge is the promises themselves — those become obligations on the fold implementations, and that is the third group.

34.4 The fold promises are arithmetic, so they go to Lean

The counting promises are elementary. The rotation promise is not, and it is the one the miss queue’s bound rests on: *does a pointer advancing by one reach every position within `depth` steps?*

That question cannot be asked of the solver, and the reason is structural rather than incidental. **Clingo grounds**: `depth` must be a number before anything reasons about it. A fact about all depths is therefore not a hard problem for the solver — it is not a problem the solver can be given.

So it goes where the route’s arithmetic always goes. `lib/lean/RouteLean/Rotation.lean` proves it for every `n`, in the shape the design actually uses:

```

def after (n p k : Nat) : Nat := (p + k) % n          -- the pointer after k answers

theorem after_succ (n p k : Nat) : after n p (k + 1) = (after n p k + 1) % n
theorem reaches (n p e : Nat) (hp : p < n) (he : e < n) : k, k < n → after n p k = e

```

`after_succ` ties the definition to the design’s own update, so the lemma is about the arbiter that exists rather than an idealised one; `reaches` is the bound. Standard axioms, no `sorry`. This is the same split the CAM uses (`RouteLean.Cam.comparator_is_equality`): an arithmetic truth proven once for all parameters in Lean, borrowed by the control layer in ASP.

Tightness — whether some position really needs all `n` steps — is deliberately **not** proven, and the file says so, because the specification does not depend on it and an unproven claim should not be left where someone might cite it.

34.5 What is done and what is owed

Done: the classification above, and the rotation lemma. The bound in the miss queue’s specification now has a proof of the fact it rests on, at every depth.

Owed, and it is the route’s oldest outstanding capability: **the window certificate itself** — a generic-slot design, the three fold interfaces as promise-constrained summaries, and a drift check pinning those interfaces to what the generator actually builds. Without the drift check the window certificate would prove things about a queue nobody wrote, which is the failure mode of every model that is allowed to differ from its subject.

Until it exists, the honest statement about any parametric entry is the one this entry now makes: **the local and pairwise promises are proven at the depth the certificate runs, and are believed at every depth for reasons stated but not machine-checked; the rotation the bound rests on is proven for every depth in Lean**. Saying that plainly is worth more than a table of green runs at four, eight and sixteen, which would look like more evidence and be less.

34.6 N copies of one unit: the stencil, the lift, and the tie

The depth question has a sibling that is easier to recognise and just as easy to hand to the wrong engine: **a block that is N copies of one unit, wired by a regular relation**. A grid of cells, a lane of identical stages, a ring of arbiters, a tree of compressors. The property is stated per unit and quantified over `N`, and the temptation is to write that quantifier in the contract and hand it to the grounder.

The grounder will instantiate all `N` copies and search over their product, so the induction over time is done `N` times over; at `N = 256` it does not finish, and the time it spends looks exactly like a scaling problem when it is a wrong-engine

problem. Conway’s Life on a 16×16 torus is the worked case: it needs induction in time, in x and in y , and only the first is clingo’s.

The split is the one this chapter already uses for depth, with the three pieces named:

1. **The unit, in clingo.** One instance with its neighbours as FREE INPUT PORTS, so the time induction ranges over every neighbourhood. This is small and fast (two seconds for a cell and its eight neighbours) and it is a real certificate, falsifiable at $T = 0$.
2. **The lift, in Lean.** “Anything true of the unit for all inputs is true at every position of every size” — one lemma, for every N , no axioms. This is the quantifier over x and y (or over lanes, or over depth), and it is the thing clingo cannot be given.
3. **The tie, mechanically.** That the generated design at the built size IS N wired copies of the unit: one template, validated wiring, no unit reading another’s next value. A script over the generator’s output, not a proof.

The decision is made at the signature, before any contract is written, and the skill asks it there: *is the block N copies of one unit over a regular relation?* If so, specify the unit. Do not write a contract that quantifies over N cells and hand it to the grounder.

And write the structure with the axes it has (Chapter 27.3). Two-dimensional lanes are not a convenience: they are how the structure the argument needs, locality and uniformity, reaches the grounder and the induction as terms instead of being reconstructed from arithmetic that hides it. They make the grid writable, printable and round-trippable at any side; they do not make the 256-cell certificate finish — that stays the unit, the lift and the tie.

35. The controlled-English surface: the sentence patterns, frozen

The specification language’s notation asked its author to think like a logician — scoped witnesses, `exists(E)`, `exactly(1, ...)` — while the behaviour being described is natural: an entry is a line fetch, a demand allocates or lifts, a prefetch merges, a second demand stalls. The user’s draft of the miss queue in structured English read better than the symbolic file and, in places, WAS a better specification. This chapter freezes that surface into language law.

The stance, first. This is a CONTROLLED grammar, not English. Free natural language is the disease this route exists to cure — Chapter 33 exists because sentences that read fine mean nothing precise. Every pattern below has exactly one parse and one meaning, defined in the same single-source grammar and lowered to the SAME core the emitter already consumes; anything outside the patterns is refused by name. The controlled surface is the PRIMARY authoring notation; the symbolic form remains the desugared core, dumpable

for inspection. This is the **phrasing**: mechanism’s own rule — sugar with a canonical form behind it — promoted to the whole language.

The sigil rule (the user’s ruling, 2026-08-31). A *sigil* is a small mark glued to the front of a word to flag it as special — here it is the `@` character, so `@when` is the sigiled spelling of `when`. The vocabulary has two kinds of word, and you can see the difference on the page. The words that give a sentence its SHAPE — the clause openers, pivots, modals and temporal anchors: `@when`, `@then`, `@given`, `@while`, `@once`, `@must`, `@never`, `@may`, `@within`, `@next`, `@every`, `@exactly`, `@still`, `@choose`, `@before`, `@during`, `@eventually` — are written with an `@` sigil, always. Everything else — articles, connectives, verbs, the words that simply read as English — is written plain, because `@a @valid request @arrives` would rebuild the logician-look this surface exists to escape, and articles ARE quantifiers here. Seventeen small marks buy two things: a reader sees a sentence’s skeleton at a glance, and a typo fails AS a keyword — a bare `when` is refused with the exact spelling to use, and an unknown `@wehn` is refused by name instead of silently becoming prose. The sigils are stripped before desugaring, so the generated core is untouched by the rule; **keywords** prints the vocabulary with the structural words marked. The patterns below are spelled as they must be written.

And the sigil stops at the surface. The `@` mark lives in the `.cn1` file alone: it is stripped at desugaring and must never appear in any GENERATED name — not in the core, not in the contract’s atoms, not in the design’s nets, and not in the printed RTL’s identifiers. A sigil in a generated name would smuggle the surface’s markup into layers that have their own naming rules, and it reads as confusion the moment an engineer opens the file. One look-alike to know about so it is never “fixed”: the `@` in printed SystemVerilog’s `always_ff @(posedge clk)` is SystemVerilog’s own event-control syntax, required by the language and unrelated to the sigil.

The pattern inventory, verbatim from the single source. The block below is rendered from the surface section of `lib/dsl/grammar.ebnf` — the same lines the desugarer compiles its matchers from, in the same order (first match wins, and that priority is now visible here instead of buried in code). The **ex:** lines are executable: the gate runs every one of them against the derived matcher and fails on any difference, and a pattern with no example refuses to build. Drift between this block and the file is a build error. The tables in 35.1–35.5 that follow are the WORD-CLASS map — what each kind of word means; this block is the sentence inventory itself.

```

SKEYWORDS ::= "@when" | "@then" | "@given" | "@while" | "@once" | "@must" | "@never"
            | "@may" | "@within" | "@next" | "@every" | "@exactly" | "@still"
            | "@choose" | "@before" | "@during" | "@eventually" | "@always"
GKEYWORDS ::= "and" | "or" | "either" | "but" | "not" | "cycle" | "cycles" | "no" | "the"
            | "a" | "an" | "one" | "more" | "such" | "fairly" | "among" | "arrives"
            | "exists" | "exist" | "accepted" | "accept" | "refuse" | "create" | "end"
            | "drive" | "send" | "answering" | "answers" | "with" | "for" | "has" | "is"

```



```

    | "are" | "becomes" | "stops" | "remembers" | "as" | "its" | "reset"
    | "equals" | "equal" | "exceed" | "belong" | "presented" | "taken" | "valid"
    | "ready" | "high" | "low" | "keep" | "existing" | "gone"

# -- surface terminals (a terminal reference captures; inline quoted words do not)
SVAR      ::= /[A-Z]\w*/
SIFACE    ::= /[a-z]\w*/
SW        ::= /\w+/
SADJ      ::= /[\w -]*?/
STOK      ::= /\S+/
SNUM      ::= /\d+/
SREST     ::= /.*/
SADJW     ::= /[\w-]+/
SCLASS    ::= "demand" | "prefetch"
SVARFIELD ::= /([A-Z]\w*)\.(\\w+)/
SEXPRTOK  ::= /\w+(?[\^ ]*\)?/
SCMP      ::= /.*(?:==|!=|<=|>=|<|>).*/
SBARE     ::= /[a-z]\w*/

# -- the condition patterns, in match order (first match wins)

condValidClassRequestIsRepeat ::= [ ("a"|"an") ] "valid" SCLASS "request" SVAR "is" [ "a" ]
# [block-vocabulary] the request is a repeat of an already-demanded line
# ex: "a valid demand request R is a repeatDemand" => request.valid(R) && repeatDemand(R)

condValidClassRequestArrives ::= [ ("a"|"an") ] "valid" SCLASS "request" SVAR "arrives"
# [block-vocabulary] the request interface's valid role plus the class field
# ex: "a valid prefetch request R arrives" => request.valid(R) && R.isDemand == 0

condClassArrives ::= [ ("a"|"an") ] SCLASS SVAR [ "for" STOK ] "arrives"
# [block-vocabulary] a classed request, optionally pinned to an address
# ex: "a demand R for E.address arrives" => request.valid(R) && R.isDemand == 1 && R.address == E.address
# ex: "a prefetch R arrives" => request.valid(R) && R.isDemand == 0

condNonRepeatDemandValid ::= [ ("a"|"an") ] "non-repeat" "demand" SVAR "is" "valid"
# [block-vocabulary]
# ex: "a non-repeat demand R is valid" => request.valid(R) && R.isDemand == 1 && !repeatDemand(R)

condIfaceValidNotAccepted ::= SIFACE SVAR "is" "valid" "but" "not" "accepted"
# valid held while the ready side refuses -- any readyValid interface
# ex: "request R is valid but not accepted" => request.valid(R) && !accepted(R)

condStillValidUnchanged ::= SVAR "@must" "@still" "be" "valid" "and" "unchanged"
# [block-vocabulary] persistence of the request payload (the interface word is implicit)
# ex: "R @must @still be valid and unchanged" => request.valid(R) && $stable(R)

```

```

condIfaceVarArrives ::= SIFACE SVAR "arrives"
#   an interface presents a payload -- generalized from the hardcoded `fill`
#   ex: "fill F arrives" => fill.arrives(F)

condIfaceArrives ::= SIFACE "arrives"
#   a payloadless interface fires -- generalized from the hardcoded `redirect`
#   ex: "redirect arrives" => redirect.arrives

condNoIface ::= "no" SIFACE
#   the interface does not fire this instant
#   ex: "no redirect" => !redirect.arrives

condIfaceTaken ::= [ ("a"|"an") ] SIFACE "for" "address" SVAR "is" "taken"
#   the handshake completes -- generalized from the hardcoded `memoryRequest`
#   ex: "a memoryRequest for address A is taken" => memoryRequest.taken(A)

condFillEventually ::= [ ("a"|"an") ] SIFACE "for" STOK "@must" "@eventually" "arrive"
#   [block-vocabulary] the obligation form; the witness variable F and the address field
#   are this block's words
#   ex: "a fill for E.address @must @eventually arrive" => s_eventually(fill.arrives(F) && R

condEntryWithAddressExists ::= [ ("a"|"an") ] SADJ ~ "entry" "with" "address" STOK "@must" "exist"
#   a witness entry, adjectives narrowing the population
#   ex: "an inFlight entry with address F.address @must exist" => some entry X1 where inFlight(X1) && X1.address == F.address

condEntryThatVerbExists ::= [ ("a"|"an") ] "entry" "that" SW "for" STOK "@must" "exist"
#   ex: "an entry that wantsFetch for A @must exist" => some entry X1 where wantsFetch(X1) && X1.address == A

condNoEntryHas ::= "no" SADJ ~ "entry" "has" STOK
#   ex: "no live entry has R.address" => !some entry X1 where exists(X1) && X1.address == R.address

condNoEntryVerb ::= "no" SADJ ~ "entry" SW
#   ex: "no live entry wantsFetch" => !some entry X1 where exists(X1) && wantsFetch(X1)

condEntryHas ::= [ ("a"|"an") ] SADJ ~ "entry" "has" STOK
#   ex: "a joinable entry has R.address" => some entry X1 where joinable(X1) && X1.address == R.address

condOneOrMoreExist ::= "one" "or" "more" SADJ ~ ( "entry" | "entries" ) "exist"
#   ex: "one or more filled demanding entries exist" => some entry X1 where filled(X1) && demanding(X1)

condIsDemand ::= SVAR "is" [ "a" ] "demand"
#   [block-vocabulary]
#   ex: "R is a demand" => R.isDemand == 1

condIsPrefetch ::= SVAR "is" [ "a" ] "prefetch"
#   [block-vocabulary]

```

```

#   ex: "R is a prefetch" => R.isDemand == 0

condIsRepeat ::= SVAR "is" [ "a" ] "repeatDemand"
#   [block-vocabulary]
#   ex: "R is a repeatDemand" => repeatDemand(R)

condIsNotRepeat ::= SVAR "is" "not" "a" "repeatDemand"
#   [block-vocabulary]
#   ex: "R is not a repeatDemand" => !repeatDemand(R)

condExists ::= SVAR [ "@still" ] "exists"
#   ex: "E @still exists" => exists(E)

condIsNotAdj ::= SVAR "is" [ "@still" ] "not" SADJW
#   ex: "E is not demanding" => !demanding(E)

condIsAdj ::= SVAR "is" [ "@still" ] SADJW
#   `non-` negates, per the adjective rules
#   ex: "E is inFlight" => inFlight(E)
#   ex: "E is non-demanding" => !demanding(E)

condBareIsAdj ::= "is" SADJW
#   anaphoric: the current entity
#   ex(entity=E): "is filled" => filled(E)

condVarVerb ::= ( "it" | SVAR ) SW
#   ex(entity=E): "it wantsFetch" => wantsFetch(E)
#   ex: "E wantsFetch" => wantsFetch(E)

condWhile ::= "@while" SREST
#   ex: "@while E is inFlight" => inFlight(E)

condMustBeGone ::= SVAR "@must" "be" "gone" [ "@next" "cycle" ]
#   ex: "E @must be gone @next cycle" => !exists(E)

condIfaceReady ::= SIFACE "@must" "be" "ready"
#   generalized from the hardcoded `request`
#   ex: "request @must be ready" => request.ready

condIfaceHigh ::= SIFACE "is" "high"
#   generalized from the hardcoded `fetchStall`
#   ex: "fetchStall is high" => fetchStall.high

condIfaceMustNotValid ::= SIFACE "@must" "not" "be" "valid"
#   generalized from the hardcoded (memoryRequest|forward)
#   ex: "memoryRequest @must not be valid" => !memoryRequest.valid

```

```

condIfaceLow ::= SIFACE "@must" "be" "low"
# generalized from the hardcoded `fetchStall`
# ex: "fetchStall @must be low" => !fetchStall.high

condMustBeNum ::= SW "@must" "be" SNUM
# ex: "liveEntries @must be 0" => liveEntries == 0

condFieldEquals ::= SVARFIELD [ "@must" ] ( "equals" | "equal" ) STOK
# ex: "D.tag @must equal E.tag" => D.tag == E.tag

condExprEquals ::= SEXPR TOK [ "@must" ] ( "equals" | "equal" ) STOK
# ex: "E.lineData @must equal D.data" => lineData(E) == D.data

condBareName ::= SBARE
# a nullary definition or counter
# ex: "fetchLimitAllows" => fetchLimitAllows

condComparison ::= SCMP
# an already-symbolic comparison passes through
# ex: "inFlightCount <= maxOutstandingFetches" => inFlightCount <= maxOutstandingFetches

# -- declaration shapes (commentary: implemented by dsl/cnl.py's declaration handlers, the
# -- way the core's declaration shapes below are implemented by the structural pass; the
# -- @property whole-sentence forms are [block-vocabulary] to a large degree and owed the
# -- same generalization the conditions received)
#
# block      ::= "@" KINDWORD NAME body-lines...
# defineShape ::= VAR "is" [ "a" ] NAME "when" condition-lines
# assumeShape ::= ( "for every entry" VAR | "@while" cond | "@when" cond ) then-line
# behaviorShape ::= trigger-lines "then" effect-lines
#                  ("is ready @exactly @when" is the biconditional shape)
#
# THE EFFECT LINES, in the order they are tried. Note the asymmetry this file cannot
# hide: the CONDITION patterns below are the single source -- cnl.py compiles its
# matchers from them and a production without a handler is refused at import -- while
# these EFFECT shapes are documented here and dispatched in code. So this list is a
# description that can drift, and the gate does not catch it; extending the derivation
# to the effects is an open item (AUTOMATION, "the effect surface").
#
# "accept" VAR          the handshake takes it
# VAR "is not accepted" and refuses it
# "fetchStall is high"  [block-vocabulary] a named port held high
# "keep the existing entry" an explicit NO-OP: lowers to nothing, because
#                           the frame monitors are what make "and nothing
#                           else happened" true

```

```

# "end" VAR "@next cycle"          the object ceases to exist
# "create entry" VAR "for address" TOK [ "with tag" TOK ]
# "@next cycle" clause             (see below)
# "choose one" ADJ "entry" VAR     a witness -- which one is the design's freedom
# "choose exactly one" [ "such" ] "entry" VAR
#                                 ...with its own effect lines under it, each a
#                                 "send forward" or an "end VAR @next cycle"
# "choose fairly among" REST       the fairness OBLIGATION, recorded (Phase 3)
# "drive" IFACE "with" TOK         the output command
# "@every" ADJ "entry stops being" ADJW "@next cycle"
#                                 a per-object effect stated in one line
# "the" ADJ "entry" VAR "that" VERB "for" TOK | "the" ADJ "entry" VAR "with" TOK
#                                 a SCOPED effect block: the description selects
#                                 the object, the lines under it are its effects
#
# and the "@next cycle" clause is one of
#
# "end" VAR                        destruction
# VAR "." FIELD "is" TOK           capture into an object's field
# NAME [ "[" expr "]" ] ( "is" | "=" ) expr
#                                 a declared @state assigned -- scalar or one
#                                 POSITION of an indexed one, with an EXPRESSION
#                                 on the right so a counter can advance.
#                                 Recognised only when NAME is a declared @state,
#                                 so an ordinary condition is still a condition
#                                 a flag set (or cleared, when negated)
#
# any condition
# scenarioShape ::= trigger-lines arrow expectation-lines
# quantBlock ::= "@every" KIND VAR ":" ( claim-lines | trigger "then" effect-lines )
# -- any domain declared @index, not only the built-in `entry`, and in a
# @behavior as well as a @property: a per-position EFFECT ("for every
# bit, hold it unless this is its turn") is as ordinary as a
# per-position claim, and having only the second is half a construct
#
# Since 2026-09-02 the surface also carries, all of them things the first block never
# needed and the second could not do without:
#
# * `@state <name> : enum { a, b, c }` -- and a member is a VALUE, so a phase is
# compared by name rather than by a number nobody can read;
# * `@next cycle <state> is <value>` / `=<expression>` -- a scalar state assigned, and
# an expression on the right, so a counter can advance. THE TARGET is next cycle and
# every READ is at the event's instant, subscripts included;
# * `@index <name> : <extent>` as a declaration of this notation, not only of the core;
# * `<port>[J]` -- element J on a port that declares `elements`, bit J on a flat one,
# the DECLARATION deciding which and never inference;
# * `@then @next cycle <condition>` in a @property, which a @scenario already took --
# the same clause, and only the declaration keyword had differed. Every consequent of

```

```

#   one claim or none: a claim is about one instant;
#   * `@always <condition>` -- a property with no trigger, the eighteenth structural keyword
#   The core had `always <expr>` from the start; the surface reached it only inside an
#   `@every ... :` block, so a top-level "the state is always one of these" needed a
#   tautological @when. Explicit and sigiled, not "a bare condition means always";
#   * a clause NAMING THE RESET is reset-exempt however it is spelled (Chapter 33), in a
#   @behavior as well as a @property. Keying that to one phrase left the other spellings
#   compiling into monitors that cannot fire.

```

35.1 Articles and quantity — the words that carry quantifiers

pattern	meaning	refused
a/an X (in a requirement)	a witness: some X must exist / satisfy	—
a/an X (in a @when trigger)	PER-INSTANCE: the behaviour speaks about every such X — “@when an entry is filled and not demanding, end it” means each one, and reading it as a witness would let one ending discharge all. The witness reading in an effect needs the explicit @choose	—
@every X	each X — universal	—
no X ...	!some X ...	—
the X that P	a DEFINITE DESCRIPTION: the unique X with P — legal only when a stated property licenses uniqueness (the miss queue’s the entry that wantsFetch for A is licensed by oneEntryPerLine)	the with no uniqueness license — refused naming the missing property
one or more X exist	some X as a proposition	—
two different X, Y	two binders with X != Y	—

pattern	meaning	refused
such X	anaphor to the NEAREST preceding X-description in the same declaration	such with no antecedent description

35.2 Time — the words that carry instants

pattern	meaning	refused
@when P	the trigger/antecedent, at T	—
@then ...	the consequent/effects	—
@next cycle (leading or trailing)	##1 on the clause it attaches to	—
@within the @next N cycles	##[1:N]	a bare @eventually with no bound, ranking, or work-conservation — the standing refusal in a @property
@must @eventually	s_eventually — an OBLIGATION (assumptions only)	—
@while P	a state condition at the same instant	—
@once P, ... @before it ends	the antecedent's onset, persisting with the lifetime escape (... !exists(E))	@once/until spans with no lifetime escape on an object — the 10b rule
@still X	persistence: X held before and holds now	@still in a first mention
@during reset	the reset exemption: judged where disable iff would silence it	—
naming the reset in any clause (@when reset == 1, ...)	the SAME exemption — it follows the reset being named, not one phrase (§33)	—
@then @next cycle Q (in a @property)	 => — the claim is about the following cycle	@next cycle on some consequents and not others: a claim is about ONE instant
@every KIND VAR : (in a @behavior)	the effects apply at every position of a declared domain	quantifying over a declared domain AND an entry at once

pattern	meaning	refused
@every KIND VAR : (in a @scenario)	the story is reached only when EVERY position satisfies the expectation — never when one happens to	a universal scenario whose SITUATION mentions the position (say the situation without it, the expectation with it)

35.3 State and effect — the verbs

pattern	meaning	refused
E is X (in a condition)	the window: $x(E)$	—
E is X / becomes X (in an effect)	##-scoped $x(E)$ at the effect's instant	—
E stops X-ing / is no longer X	$!x(E)$ at the effect's instant	—
E.f is V (in an effect)	CAPTURE — identity, the very token flows	—
remembers V as its W	capture into the value window: $w(E) = V$	—
E.w (w not a payload field)	the value vocabulary: $w(E)$ — $E.lineData$ is $lineData(E)$; only address and tag are entry attributes	—
end E / E must be gone	$!exists(E)$	—
keep the existing entry	an explicit no-op: lowers to NOTHING — the frame monitors are what make “nothing else happened” true	any other effect in the same sentence
R is accepted / is not accepted	the handshake held / the ready wire refused	—
X arrives	the interface presents: its valid role, whatever the protocol	—
answers E	the correspondence window (answering E)	—
has address A	address equality, through the sort system (opaque = the theory)	—

35.3a State that is not an object

The miss queue’s state is *objects* — entries with fields — and for a long time the surface could describe nothing else. A block whose state is a scalar FSM phase, a counter and an indexed array needs four things the language now has:

pattern	meaning
@state phase : enum { idle, receiving, ... }	a phase whose members are values : phase == presenting compares the name, not a number
@next cycle phase is receiving	a scalar state ASSIGNED in an effect
@next cycle bitIndex = bitIndex + 1	...whose right-hand side may be an expression, so a counter can advance
@index bit : dataBits · @every bit J:	a domain the block declares, and a quantifier that ranges over it

An effect has a target and a source, and they are read at different instants. The left-hand side is what the window holds NEXT cycle; everything read on the right is taken at the instant the event happened — including a subscript, because which position an effect writes is decided by the index *now*, not by what the index will hold afterwards. That is a flip-flop, spelled out: **##1 bitIndex = bitIndex + 1** means the counter advances, and reading both sides at one instant would say “next cycle’s counter is next cycle’s counter plus one”, which nothing can satisfy.

A domain must be declared, whether a SUBSCRIPT or a QUANTIFIER names it, and a subscript ranging over a domain of the wrong extent is reported as well. All three are otherwise satisfiable, and a satisfiable contract that claims the wrong thing is the failure this route works hardest to prevent — the quantifier being the sharpest case, since an undeclared kind is read as an OBJECT kind, and a claim quantified over a population no linkage can mount is not wrong but VACUOUS: it is counted among the monitors and cannot fire.

35.4 Requirement and connective words

pattern	meaning	refused
@must P	the requirement marker: P is the checked consequent	—
@must @never P	always !P	—
@must not exceed N	always <= N	—
@never more than one of:	pairwise mutual exclusion over the listed states	—

pattern	meaning	refused
and / or / either ... or	&& / \ \	—
but not P	&& !P	—
@exactly @when P	a BICONDITIONAL: both directions in one construct — replacing a behaviour/property pair kept in agreement by hand	—
already	NOISE — carries no meaning and is refused, because a word that sometimes means something and sometimes does not is how misreadings breed. (@still means persistence; already means nothing has does not.)	always

35.5 Choice and fairness — the constructs that are NOT sugar

pattern	meaning
@choose one X	the witness: the design serves SOME scope-satisfying object; which one is implementation freedom (33.2's witness binder as a verb)
@choose @exactly one X	the witness plus the two-distinct-served failure
@choose fairly among Xs	THE FAIRNESS OBLIGATION, named where it arises: every persistently-eligible object is served within the population's extent. It is what per-object bounds rest on (Chapter 34), it ties to the parametric fact in Lean (RouteLean.Rotation.reaches), and stating it in the behaviour makes the dependency visible in the source instead of in a comment
@may P (in a scenario)	reachability: P must remain possible

pattern	meaning
@may P @before Q	a reachable ordering: an execution exists where P precedes Q

35.6 The corpus gate

The user’s structured-English draft of the miss queue is the surface’s REFERENCE CORPUS. The acceptance bar, fixed before any code: compiled, it must produce the SAME certificate verdicts as the entry’s specification of record — at the time the hand-written symbolic `rvMissq.spec`, since archived when the English became the notation of record; today the gate holds the compiled English to the contract of record’s verdicts — the real design green, the sabotaged designs red, each through the same monitors. Two spellings, one block, one truth: a differential, which is this route’s kind of evidence.

35.7 The pipeline, and the gate on every arrow

The chain, written as what a person does and what a machine checks. Read it top to bottom: each artifact is derived from the one above it, and every arrow either has a mechanical gate or is named as human work with its handrail.

arrow	who does it	the gate
English $\rightarrow$ <code>.cnl</code>	a person (or a model, read back)	the traceability table , read both ways by test_v2_cnl_traceability : every promise names its declaration, every declaration is owned by a promise, and honesty markers — WITHDRAWN, NOT TRANSCRIBED, SIGNATURE — are stated, never silent. Plus the ladder: a person approves the rung
<code>.cnl</code> $\rightarrow$ <code>.cnl.core</code>	the desugarer, mechanically	the committed core must equal what the corpus desugars to — drift is a failing gate, and the diff is readable

arrow	who does it	the gate
<code>.cnl.core</code> $\rightarrow$ contract	the emitter, mechanically	refusals by name (nothing lowers silently); the generality gate runs BOTH corpus blocks; the Stage-5 differential — an independent evaluator of the same claims over random traces must agree with the generated contract under clingo on every (monitor, instant) verdict, sabotages caught — with the schema meanings themselves proven faithful in Lean (<code>RouteLean/Claims.lean</code>)
contract $\rightarrow$ certificate	clingo	the corpus gate: the real design green with the whole set inductive, and both sabotage families red — the claims-level defect and the behaviour-level one the frames catch
certificate $\rightarrow$ RTL	the printer and the round trip	the existing harness: the authored model, the translated print and a simulator (Verilator or Icarus) agreeing value for value

Two of these gates have already paid for themselves on their first run. The traceability gate found two claims no English sentence owned — `oneEntryPerLine` and `entryPhasesDoNotOverlap`, both true, both load-bearing, both stated nowhere a person could point to — and the English gained P16 and P17. And the corpus gate is what held the whole surface build honest: the English was not done until it produced the same verdicts as the symbolic reference, sabotages included.

36. The proof discipline: induction, not simulation, in both engines

This chapter states one rule that the rest of the book has been obeying without naming it, and that the multiplier entry (2026-09-05) forced into words when it hit the same wall in the solver and in the prover on the same day. The user’s ruling: *“The key in both Lean and ASP, we should not simulate directly. It will make the proofs blow in large designs. Induction is key in both... and think like math proofs and not simulating proofs.”*

The rule. A proof is an argument that covers every case by the structure of the thing — by induction over its bits, its digits, its rows, its levels, its instants — and never by enumerating, evaluating or unrolling the cases. The reason is the same in both engines. A design’s cases grow exponentially with its size: a 64-bit operand has 2^{64} values, an eight-level compressor tree written out as a term carries 3^8 copies of every row, thirty-two free control bits fork a term family 2^{32} ways. A proof built by simulation grows with the cases; a proof built by induction does not grow at all. So the question to ask of any step that is slow, or large, or that will not finish, is what it is enumerating — and the answer is always followed by the same move, which is to find the induction it was standing in for.

In the solver. Clingo grounds: whatever a rule ranges over becomes as many instances as there are values. That is why the route never enumerates data (Chapter 17: opaque roles, tokens, a free value per distinct term at a boundary), why the proof is the induction step in normal form (Chapter 16), why the bounded legs are kept short and pinned to one control corner per solve, why a structure the leg cannot hold is replaced by its meaning through an obligation view whose equivalence is owed to Lean (Chapter 24), and why the budget rule (Chapter 26) is a measurement and not a hope. Two corollaries the multiplier paid for: a view’s dropped cone must be computed jointly over all views and must count only live readers, or a shared or a dead net keeps the structure alive; and a control bit computed from data is a free boundary that forks, so spell it as a data term where you can.

In the prover. The Lean kernel evaluates when asked, and asking it about the design is the same mistake. Never `decide`, `rfl`, `native_decide` or `by_decide` on a term that contains the design; never a modulus of 2^{128} inside `omega`; never a bare `simp` on a goal that carries such a numeral. Convert to numbers once at the boundary — a word is its natural, a signed word’s value is $x - x_{\{w-1\}} \cdot 2^w$ — and work below it in the naturals and integers, where a row identity is `ring` after a case split on one sign bit and the sum of the rows is a congruence calculation. State every structured fact generically — digits as a sum with a telescoping invariant, a tree as a list of ranged rows with one lemma per level, an adder as spans that compose with an invariant over the levels — and obtain the instance as a corollary. The genuinely bit-level facts are few, each stated once over naturals. Constants may be evaluated; the design may

not.

Paper first. The proof is worked out completely on paper before any Lean: every lemma stated and proven, the lemma graph drawn, the connectivity checked so that each conclusion is a premise of the next, and every identity checked numerically against the design's own definitions. Lean then transcribes. And the transcription's definitions are the design's nets by name, evaluated against the design file before any proof — a model written from a reading of the design is a third copy of it.

Prove that the algorithm computes the specification. Never that it equals the previous machine: that proves agreement, not correctness, and makes every level hostage to every earlier one.

Where simulation is used. As a witness, never as evidence for a theorem: the round trip checks that the print says what the design says; the transcription checks establish that what was transcribed is the file; a bounded leg's counterexample shows a person where a property failed. Each catches an error a proof about the wrong definitions could not; none stands in for the induction, and no theorem cites one.

The learnings document carries this rule with the measurements behind each clause; the operating skill carries it as a checklist.